

STensor/TensorComponents

Setup

```
In[1]:= << mGRG`STensor`

In[2]:= $PreRead = ReplaceAll[#,
    expr_String => StringReplace[expr, "SP" -> "mGRG`STensor`Private"]] &;

In[3]:= DefKind[Greek, Alphabet["Greek"]];
DefKind[Capital, ToUpperCase /@ Alphabet[]];
DefKind[Bar, "b" <> # & /@ Alphabet[]];
DefKind[Dot, "d" <> # & /@ Alphabet[]];
DefKind[Hat, "h" <> # & /@ Alphabet[]];
```

Component Operations

```
In[*]:= Tdefine[T, "ba"]; Tdefine[F, "-ba"]; Tdefine[v, 1]; Tdefine[ξ, 1]; Tdefine[f[]]
```

CDtoBD

```
In[*]:= CDtoBD::usage
Out[*]:= CDtoBD[expr, covD] expands all covariant derivatives 'covD' in
    'expr' into base derivatives and connection coefficients. If
    'covD' is omitted, the default covariant derivative CD is used.
```

Covariant derivative를 BD와 Affine connection 계수를 이용한 표현으로 변환한다. 옵션으로 공변도함수의 이름이 있다.

$$\nabla_a T_b^c = D_a T_b^c - \Gamma_{ab}^p T_p^c + \Gamma_{ap}^c T_b^p$$

```
In[*]:= CD[la, F[lb, lc]]
Out[*]:= CDtoBD[%]
```

$\nabla_a F_{bc}$

```
Out[*]:= ∂aFbc - Fdc Γabd - Fbd Γacd
```

```
In[*]:= CD[la, lb, v[uc]] - CD[lb, la, v[uc]]
CDtoBD[%]
CDtoBD[%]
```

```
Out[*]=
```

$$\nabla_a \nabla_b V^c - \nabla_b \nabla_a V^c$$

```
Out[*]=
```

$$\partial_a \nabla_b V^c - \partial_b \nabla_a V^c - \nabla_d V^c \Gamma_{ab}^d + \nabla_b V^d \Gamma_{ad}^c + \nabla_d V^c \Gamma_{ba}^d - \nabla_a V^d \Gamma_{bd}^c$$

```
Out[*]=
```

$$\begin{aligned} & \partial_a \partial_b V^c - \partial_b \partial_a V^c - \partial_d V^c \Gamma_{ab}^d + \partial_d V^c \Gamma_{ba}^d + \partial_a \Gamma_{bd}^c V^d - \\ & \partial_b \Gamma_{ad}^c V^d - \Gamma_{ad}^e \Gamma_{be}^c V^d + \Gamma_{ad}^c \Gamma_{be}^d V^e - \Gamma_{ab}^d \Gamma_{de}^c V^e + \Gamma_{ba}^d \Gamma_{de}^c V^e \end{aligned}$$

텐서가 아닌 표현의 Lie derivative는 의미가 없으므로 LD 안에 있는 CD는 변환되지 않는다.

```
In[*]:= LD[v, CD[la, F[lb, lc]]]
CDtoBD[%]
```

```
Out[*]=
```

$$\mathcal{L}_v \nabla_a F_{bc}$$

```
Out[*]=
```

$$\mathcal{L}_v \nabla_a F_{bc}$$

옵션으로 공변 도함수의 이름이 있다.

```
In[*]:= SetDefaultKind[Capital]
```

```
In[*]:= DefDerivativeOperator[CovD, "D", Greek]
```

```
In[*]:= Tdefine[V[lμ], "a"]
```

```
In[*]:= {CovD[lμ, ξ[ua]], CovD[lμ, V[uv]]} (* ξ의 Kind는 Latin이다. *)
CDtoBD[#, CovD] & /@ %
```

```
Out[*]=
```

$$\{\mathcal{D}_\mu \xi^a, \mathcal{D}_\mu V^\nu\}$$

```
Out[*]=
```

$$\{\hat{\partial}_\mu \xi^a, \hat{\partial}_\mu V^\nu + \Gamma[\mathcal{D}]_{\mu\alpha}{}^\nu V^\alpha\}$$

```
In[*]:= UndefTensor[V]; UndefDerivativeOperator[CovD]
SetDefaultKind[Latin]
```

CommuteCD

In[*]:= **CommuteCD::usage**

Out[*]=

CommuteCD[{a, b}, expr, covD] commutes the covariant derivatives with indices 'a' and 'b' in 'expr', expressing the result in terms of the Riemann and torsion tensors. If 'covD' is omitted, the default CD is used.

Covariant derivative가 두 번 이상 연속해서 작용할 때, 원래의 표현과 같으면서 두 CovD의 인덱스를 교환시킨 표현을 준다. 첫 번째 인자는 뒤 바꿀 두 개의 인덱스로 이루어진 리스트이고, 두 번째 인자는 임의의 표현이다. 옵션으로 공변 도함수의 이름이 있다.

$$(\nabla_a \nabla_b - \nabla_b \nabla_a + t_{ab}{}^p \nabla_p) T_c{}^d = (\text{riemannSign})(R_{abc}{}^p T_p{}^d - R_{abp}{}^d T_c{}^p)$$

Torsion-Free가 아닌 경우

In[*]:= **TorsionFreeQ[CD] = False;**

In[*]:= **expr = CD[la, lb, T[lc, ud]]**

CommuteCD[{la, lb}, expr]

Out[*]=

$$\nabla_a \nabla_b T_c{}^d$$

Out[*]=

$$\nabla_b \nabla_a T_c{}^d - R_{abe}{}^d T_c{}^e + R_{abc}{}^e T_e{}^d - \nabla_e T_c{}^d t_{ab}{}^e$$

Torsion-Free인 경우

In[*]:= **TorsionFreeQ[CD] = True;**

In[*]:= **CommuteCD[{la, lb}, expr]**

Out[*]=

$$\nabla_b \nabla_a T_c{}^d - R_{abe}{}^d T_c{}^e + R_{abc}{}^e T_e{}^d$$

In[*]:= **expr = CD[la, ub, lc, F[lb, ud]]**

CommuteCD[{la, ub}, expr]

Out[*]=

$$\nabla_a \nabla^b \nabla_c F_b{}^d$$

Out[*]=

$$\nabla^b \nabla_a \nabla_c F_b{}^d - \nabla_c F_b{}^d R_a{}^b - \nabla_c F_b{}^e R_a{}^b{}^d{}_e + \nabla_b F_e{}^d R_a{}^e{}^b{}_c$$

뒤 바꿀 두 개의 인덱스는 연속해서 있어야 한다.

In[*]:= **expr**
CommuteCD[{la, lc}, expr]

Out[*]=

$$\nabla_a \nabla^b \nabla_c F_b^d$$

Out[*]=

$$\nabla_a \nabla^b \nabla_c F_b^d$$

In[*]:= **expr**
CommuteCD[{lc, ub}, expr]

Out[*]=

$$\nabla_a \nabla^b \nabla_c F_b^d$$

Out[*]=

$$\nabla_a \nabla_c \nabla^b F_b^d + \nabla_a R_c^b F_b^d - \nabla_a R_{ce}^b F_b^d + \nabla_a F_b^d R_c^b - \nabla_a F_b^e R_{ce}^b$$

In[*]:= **F[uc, le] expr**
CommuteCD[{la, ub}, %]

Out[*]=

$$\nabla_a \nabla^b \nabla_c F_b^d F_e^c$$

Out[*]=

$$\nabla^b \nabla_a \nabla_c F_b^d F_e^c - \nabla_b F_c^d F_e^b R_a^c + \nabla_b F_c^d F_e^f R_a^c - \nabla_b F_c^f F_e^b R_a^c$$

In[*]:= **CD[la, lb, v[uc]]**
CommuteCD[{la, lb}, %]

Out[*]=

$$\nabla_a \nabla_b V^c$$

Out[*]=

$$\nabla_b \nabla_a V^c - R_{abd}^c V^d$$

In[*]:= **CD[la, lb, f[]]**
CommuteCD[{la, lb}, %]

Out[*]=

$$\nabla_a \nabla_b f$$

Out[*]=

$$\nabla_b \nabla_a f$$

CD의 인덱스가 아니면 영향이 없다.

```
In[*]:= expr = CD[la, BD[ub, CD[lc, F[lb, ud]]]]
CommuteCD[{la, ub}, expr]
```

Out[*]=

$$\nabla_a \partial^b \nabla_c F_b^d$$

Out[*]=

$$\nabla_a \partial^b \nabla_c F_b^d$$

텐서 표현이 아닌 것에도 영향이 없다.

```
In[*]:= expr = CD[la, ld, BD[ub, CD[lc, F[lb, ud]]]]
CommuteCD[{lb, ud}, expr]
```

Out[*]=

$$\nabla_a \nabla_b \partial^d \nabla_c F_d^b$$

Out[*]=

$$\nabla_a \nabla_b \partial^d \nabla_c F_d^b$$

옵션으로 공변 도함수의 이름이 있다.

```
In[*]:= SetDefaultKind[Capital];
DefDerivativeOperator[CovD, "D", Capital, TorsionFreeQ -> False];
Tdefine[V[la], 1]
```

```
In[*]:= {CovD[la, lb, xi[ua]], CovD[la, lb, V[uc]]} (* xi의 Kind는 Latin이다. *)
CommuteCD[{la, lb}, #, CovD] & /@ %
```

Out[*]=

$$\{\mathcal{D}_A \mathcal{D}_B \xi^a, \mathcal{D}_A \mathcal{D}_B V^c\}$$

Out[*]=

$$\{\mathcal{D}_B \mathcal{D}_A \xi^a - \mathcal{D}_C \xi^a t_{AB}^C, \mathcal{D}_B \mathcal{D}_A V^c - \mathcal{D}_D V^c t_{AB}^D - R[\mathcal{D}]_{ABD}^C V^D\}$$

```
In[*]:= UndefTensor[V]; UndefDerivativeOperator[CovD];
SetDefaultKind[Latin]
```

GammaToMetric

```
In[*]:= GammaToMetric::usage
```

Out[*]=

GammaToMetric[expr, covD] expresses Christoffel symbols (connection coefficients) in 'expr' in terms of derivatives of the metric. If 'covD' is omitted, the default CD is used.

임의의 표현에 있는 metric-connection Γ_{ab}^c 를 계량 텐서 표현으로 변환한다. 옵션으로 공변 도함수의 이름이 있다.

$$\Gamma_{ab}^c = g^{cd} \Gamma_{abd} = \Gamma_{ba}^c \mid \text{MTW, Wald}$$

$$\Gamma_{abc} \equiv \frac{1}{2} (D_a g_{bc} + D_b g_{ac} - D_c g_{ab} + f_{abc} + f_{cab} + f_{cba} + t_{abc} + t_{cab} + t_{cba})$$

Torsion-Free가 아니고 Non-coordinate basis에서

```
In[*]:= TorsionFreeQ[CD] = False;  
Off[CoordinateBasisFlag]
```

```
In[*]:= GammaToMetric[GammaCD[la, lb, lc]]
```

```
Out[*]=
```

$$\frac{1}{2} \hat{\partial}_a g_{bc} + \frac{1}{2} \hat{\partial}_b g_{ac} - \frac{1}{2} \hat{\partial}_c g_{ab} + \frac{1}{2} f_{abc} + \frac{1}{2} f_{cab} + \frac{1}{2} f_{cba} + \frac{1}{2} t_{abc} + \frac{1}{2} t_{cab} + \frac{1}{2} t_{cba}$$

Torsion-Free가 아니고 Coordinate basis에서

```
In[*]:= On[CoordinateBasisFlag]
```

```
In[*]:= GammaToMetric[GammaCD[la, lb, lc]]
```

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{bc} + \frac{1}{2} \partial_b g_{ac} - \frac{1}{2} \partial_c g_{ab} + \frac{1}{2} t_{abc} + \frac{1}{2} t_{cab} + \frac{1}{2} t_{cba}$$

Torsion-Free이고 Coordinate basis에서

```
In[*]:= TorsionFreeQ[CD] = True;
```

```
In[*]:= GammaToMetric[GammaCD[la, lb, lc]]
```

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{bc} + \frac{1}{2} \partial_b g_{ac} - \frac{1}{2} \partial_c g_{ab}$$

```
In[*]:= GammaToMetric[GammaCD[la, lb, uc]]
```

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{bd} g^{cd} + \frac{1}{2} \partial_b g_{ad} g^{cd} - \frac{1}{2} \partial_d g_{ab} g^{cd}$$

```
In[*]:= GammaToMetric[GammaCD[la, ub, lc]]
```

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{dc} g^{bd} - \frac{1}{2} \partial_c g_{ad} g^{bd} + \frac{1}{2} \partial_d g_{ac} g^{bd}$$

```
In[*]:= GammaToMetric[GammaCD[la, ub, lc]]
```

```
Out[*]=
```

$$\frac{1}{2} \partial_a g_{dc} g^{bd} - \frac{1}{2} \partial_c g_{ad} g^{bd} + \frac{1}{2} \partial_d g_{ac} g^{bd}$$

In[*]:= **GammaToMetric**[GammaCD[ua, lb, lc]]

Out[*]=

$$\frac{1}{2} \partial_b g_{dc} g^{ad} - \frac{1}{2} \partial_c g_{db} g^{ad} + \frac{1}{2} \partial_d g_{bc} g^{ad}$$

In[*]:= **GammaToMetric**[GammaCD[ua, lb, uc]]

Out[*]=

$$-\frac{1}{2} \partial_d g_{eb} g^{ae} g^{cd} + \frac{1}{2} \partial_b g_{de} g^{ad} g^{ce} + \frac{1}{2} \partial_d g_{be} g^{ad} g^{ce}$$

In[*]:= **GammaToMetric**[GammaCD[ua, ub, lc]]

Out[*]=

$$\frac{1}{2} \partial_d g_{ec} g^{ae} g^{bd} - \frac{1}{2} \partial_c g_{de} g^{ad} g^{be} + \frac{1}{2} \partial_d g_{ec} g^{ad} g^{be}$$

In[*]:= **GammaToMetric**[GammaCD[ua, ub, uc]]

Out[*]=

$$-\frac{1}{2} \partial_d g_{ef} g^{ae} g^{bf} g^{cd} + \frac{1}{2} \partial_d g_{ef} g^{ae} g^{bd} g^{cf} + \frac{1}{2} \partial_d g_{ef} g^{ad} g^{be} g^{cf}$$

Covariant derivative를 AffineCD로 변환한 후 다시 Metricg의 표현으로 변환시키는 경우

In[*]:= **CD**[la, lb, F[ua, ub]]

CDtoBD[%]

CDtoBD[%]

Out[*]=

$$\nabla_a \nabla_b F^{ab}$$

Out[*]=

$$\partial_a \nabla_b F^{ab} + \nabla_a F^{ba} \Gamma_{cb}^c$$

Out[*]=

$$\partial_a \partial_b F^{ab} + \partial_a \Gamma_{bc}^b F^{ac} + \partial_a \Gamma_{bc}^a F^{cb} + \partial_a F^{bc} \Gamma_{cb}^a + \\ \partial_a F^{ab} \Gamma_{cb}^c + \partial_a F^{ba} \Gamma_{cb}^c + F^{ab} \Gamma_{cb}^c \Gamma_{da}^d + F^{ab} \Gamma_{ba}^c \Gamma_{dc}^d$$

In[*]:=

GammaToMetric[%]

Out[*]=

$$\begin{aligned}
& \partial_a \partial_b F^{ab} + \frac{1}{2} \partial_a g^{bc} \partial_b g_{dc} F^{ad} - \frac{1}{2} \partial_a g^{bc} \partial_c g_{bd} F^{ad} + \frac{1}{2} \partial_a g^{bc} \partial_d g_{bc} F^{ad} + \frac{1}{2} \partial_a g^{ab} \partial_c g_{db} F^{cd} - \\
& \frac{1}{2} \partial_a g^{ab} \partial_b g_{cd} F^{dc} + \frac{1}{2} \partial_a g^{ab} \partial_c g_{db} F^{dc} - \frac{1}{2} \partial_a \partial_b g_{cd} F^{dc} g^{ab} + \frac{1}{2} \partial_a F^{bc} \partial_b g_{cd} g^{ad} + \\
& \frac{1}{2} \partial_a F^{bc} \partial_c g_{bd} g^{ad} - \frac{1}{2} \partial_a F^{bc} \partial_d g_{cb} g^{ad} + \frac{1}{2} \partial_a \partial_b g_{cd} F^{bc} g^{ad} + \frac{1}{2} \partial_a \partial_b g_{cd} F^{cb} g^{ad} - \\
& \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{fe} g^{ad} g^{bc} + \frac{1}{2} \partial_a \partial_b g_{cd} F^{ac} g^{bd} - \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{fe} g^{ac} g^{bd} - \frac{1}{2} \partial_a \partial_b g_{cd} F^{ad} g^{cb} + \\
& \frac{1}{2} \partial_a F^{ab} \partial_b g_{cd} g^{cd} + \frac{1}{2} \partial_a F^{ba} \partial_b g_{cd} g^{cd} + \frac{1}{2} \partial_a F^{ab} \partial_c g_{bd} g^{cd} + \frac{1}{2} \partial_a F^{ba} \partial_c g_{bd} g^{cd} + \\
& \frac{1}{2} \partial_a \partial_b g_{cd} F^{ab} g^{cd} - \frac{1}{2} \partial_a F^{ab} \partial_c g_{db} g^{dc} - \frac{1}{2} \partial_a F^{ba} \partial_c g_{db} g^{dc} + \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{eb} g^{ac} g^{df} + \\
& \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ae} g^{bc} g^{df} + \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ea} g^{bc} g^{df} + \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ab} g^{df} g^{ec} + \\
& \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ba} g^{df} g^{ec} - \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{bf} g^{ac} g^{ed} - \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{fb} g^{ac} g^{ed} + \\
& \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{fc} g^{ba} g^{ed} - \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{af} g^{bc} g^{ed} - \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{fa} g^{bc} g^{ed} + \\
& \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ad} g^{bc} g^{ef} + \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ab} g^{dc} g^{ef} + \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ba} g^{dc} g^{ef} + \\
& \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{cb} g^{ed} g^{fa} - \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ab} g^{ed} g^{fc} - \frac{1}{4} \partial_a g_{bc} \partial_d g_{ef} F^{ba} g^{ed} g^{fc}
\end{aligned}$$

In[*]:=

% // Tsimplify

Out[*]=

$$-\frac{1}{2} \partial_a g_{bc} \partial_d g^{cb} F^{ad}$$

In[*]:=

% /. BDinvRule[]

Out[*]=

$$\frac{1}{2} \partial_a g_{bc} \partial_d g_{ef} F^{ad} g^{bf} g^{ce}$$

In[*]:=

% // Tsimplify

Out[*]=

0

옵션으로 공변 도함수의 이름이 있다.

In[*]:=

SetDefaultKind[Capital]

In[*]:=

```
Off[CoordinateBasisFlag[Latin]];
DefDerivativeOperator[CovD, "D", Latin];
DefMetric[Phi, "g", Latin]
```


아직 CovD와 Phi는 서로 관련이 없다:

```
In[*]:= GammaCovD[la, lb, lc]
GammaToMetric[%, CovD]
```

```
Out[*]:=
 $\Gamma[\mathcal{D}]_{abc}$ 
```

... Msg: CovD is not metric —compatible with Phi

```
Out[*]:=
 $\Gamma[\mathcal{D}]_{abc}$ 
```

```
In[*]:= SetMetricCompatible[CovD, Phi]
```

```
In[*]:= GammaCovD[la, lb, lc]
GammaToMetric[%, CovD]
```

```
Out[*]:=
 $\Gamma[\mathcal{D}]_{abc}$ 
```

```
Out[*]:=

$$\frac{1}{2} \hat{\partial}_a \Phi_{bc} + \frac{1}{2} \hat{\partial}_b \Phi_{ac} - \frac{1}{2} \hat{\partial}_c \Phi_{ab} + \frac{1}{2} f_{abc} + \frac{1}{2} f_{cab} + \frac{1}{2} f_{cba}$$

```

```
In[*]:= ClearMetricCompatible[CovD, Phi];
UndefMetric[Phi];
UndefDerivativeOperator[CovD]
```

```
In[*]:= SetDefaultKind[Latin];
On[CoordinateBasisFlag[Latin]]
```

LDtoCD

```
In[*]:= LDtoCD::usage
```

```
Out[*]:=
LDtoCD[expr, covD] converts the Lie derivative LD in 'expr'
into covariant derivatives 'covD'. If 'covD' is omitted,
the default torsion-free covariant derivative CD is used.
```

Lie derivative를 covariant derivative로 바꾼다.

$$\mathcal{L}_v T_a^b = v^p \nabla_p T_a^b + \nabla_a v^p T_p^b - \nabla_p v^b T_a^p$$

Torsion-Free가 아닌 경우

```
In[*]:= TorsionFreeQ[CD] = False;
```

```
In[*]:= LDtoCD[LD[v, t[ua, ub]]]
```

Msg: CD is not torsion -free

```
Out[*]=
```

$$\mathcal{L}_v t[ua, ub]$$

Torsion-Free인 경우

```
In[*]:= TorsionFreeQ[CD] = True;
```

```
In[*]:= LD[v, xi[la]]
% // LDtoCD
```

```
Out[*]=
```

$$\mathcal{L}_v \xi_a$$

```
Out[*]=
```

$$\nabla_b \xi_a v^b + \nabla_a v^b \xi_b$$

When $t_{ab}^c = 0$ in coordinate basis, $\Gamma_{ab}^c = \Gamma_{(ab)}^c$. Then $\mathcal{L}_v \xi_a = v^b \partial_b \xi_a + \xi_b \partial_a v^b$.

```
In[*]:= CoordinateBasisQ[DefaultKind]
```

```
Out[*]=
```

True

```
In[*]:= CDtoBD[%]
% // Tsimplify
```

```
Out[*]=
```

$$\partial_b \xi_a v^b + \partial_a v^b \xi_b + \Gamma_{ab}^c v^b \xi_c - \Gamma_{ba}^c v^b \xi_c$$

```
Out[*]=
```

$$\partial_b \xi_a v^b + \partial_a v^b \xi_b$$

```
In[*]:= LD[v, T[la, ub]]
% // LDtoCD
```

```
Out[*]=
```

$$\mathcal{L}_v T_a^b$$

```
Out[*]=
```

$$-\nabla_c v^b T_a^c + \nabla_a v^c T_c^b + \nabla_c T_a^b v^c$$

```
In[*]:= CD[1, LD[v, T[ua, ub]]]
% // LDtoCD
```

```
Out[*]=
```

$$\nabla^1 \mathcal{L}_v T^{ab}$$

```
Out[*]=
```

$$\nabla^1 v^c \nabla_c T^{ab} - \nabla^1 T^{cb} \nabla_c v^a - \nabla^1 T^{ac} \nabla_c v^b - \nabla^1 \nabla_c v^b T^{ac} - \nabla^1 \nabla_c v^a T^{cb} + \nabla^1 \nabla_c T^{ab} v^c$$

```
In[*]:= LD[v, CD[la, T[ua, ub]]]
% // LDtoCD
```

```
Out[*]=
```

$$\mathcal{L}_v \nabla_a T^{ab}$$

```
Out[*]=
```

$$-\nabla_a T^{ac} \nabla_c v^b + \nabla_a \nabla_c T^{cb} v^a$$

```
In[*]:= LD[v, T[lb, lc] × CD[la, T[ua, ub]]]
% // LDtoCD
```

```
Out[*]=
```

$$\nabla_a T^{ab} \mathcal{L}_v T_{bc} + \mathcal{L}_v \nabla_a T^{ab} T_{bc}$$

```
Out[*]=
```

$$\nabla_b T^{bd} \nabla_c v^a T_{da} + \nabla_a \nabla_b T^{bd} T_{dc} v^a + \nabla_a T^{ab} \nabla_d T_{bc} v^d$$

변환될 CovD를 지정할 수 있다:

```
In[*]:= DefDerivativeOperator[CovD, "D", Latin]
```

```
In[*]:= expr = LD[v, T[la, lb]]
LDtoCD[expr]
```

```
Out[*]=
```

$$\mathcal{L}_v T_{ab}$$

```
Out[*]=
```

$$\nabla_b v^c T_{ac} + \nabla_a v^c T_{cb} + \nabla_c T_{ab} v^c$$

```
In[*]:= expr
LDtoCD[expr, CovD]
```

```
Out[*]=
```

$$\mathcal{L}_v T_{ab}$$

```
Out[*]=
```

$$D_b v^c T_{ac} + D_a v^c T_{cb} + D_c T_{ab} v^c$$

```
expr
LDtoCD[expr, BD]
```

```
Out[*]=
```

$$\mathcal{L}_v T_{ab}$$

```
Out[*]=
```

$$\partial_b v^c T_{ac} + \partial_a v^c T_{cb} + \partial_c T_{ab} v^c$$

참고: 비좌표 기준계에서

```
In[*]:= Off[CoordinateBasisFlag]
LDtoCD[expr]
CDtoBD[%]
% // Tsimplify
```

```
Out[*]=
```

$$\nabla_b v^c T_{ac} + \nabla_a v^c T_{cb} + \nabla_c T_{ab} v^c$$

```
Out[*]=
```

$$\hat{\partial}_b v^c T_{ac} + \hat{\partial}_a v^c T_{cb} + \hat{\partial}_c T_{ab} v^c + \Gamma_{bc}^d T_{ad} v^c - \Gamma_{cb}^d T_{ad} v^c + \Gamma_{ac}^d T_{db} v^c - \Gamma_{ca}^d T_{db} v^c$$

```
Out[*]=
```

$$\hat{\partial}_b v^c T_{ac} + \hat{\partial}_a v^c T_{bc} + \hat{\partial}_c T_{ab} v^c + \Gamma_{bcd} T_a^d v^c - \Gamma_{cbd} T_a^d v^c + \Gamma_{acd} T_b^d v^c - \Gamma_{cad} T_b^d v^c$$

참고: 좌표 기준계에서

```
In[*]:= On[CoordinateBasisFlag]
LDtoCD[expr]
CDtoBD[%]
% // Tsimplify
```

```
Out[*]=
```

$$\nabla_b v^c T_{ac} + \nabla_a v^c T_{cb} + \nabla_c T_{ab} v^c$$

```
Out[*]=
```

$$\partial_b v^c T_{ac} + \partial_a v^c T_{cb} + \partial_c T_{ab} v^c + \Gamma_{bc}^d T_{ad} v^c - \Gamma_{cb}^d T_{ad} v^c + \Gamma_{ac}^d T_{db} v^c - \Gamma_{ca}^d T_{db} v^c$$

```
Out[*]=
```

$$\partial_b v^c T_{ac} + \partial_a v^c T_{bc} + \partial_c T_{ab} v^c$$

텐서 표현이 아니거나 CovD와 부합되지 않는 것은 변환되지 않는다:

```
In[*]:= LD[v, something[la]]
% // LDtoCD
```

```
Out[*]=
```

$$\mathcal{L}_v \text{something}[la]$$

```
Out[*]=
```

$$\mathcal{L}_v \text{something}[la]$$

```
In[*]:= LD[v, BD[la, f[]]]
% // LDtoCD
```

```
Out[*]=
```

$$\mathcal{L}_v \partial_a f$$

```
Out[*]=
```

$$\mathcal{L}_v \partial_a f$$

<code>In[*]:=</code>	<code>LD[v, BD[la, f[]]]</code> <code>LDtoCD[%, BD]</code>
<code>Out[*]=</code>	$\mathcal{L}_v \partial_a f$
<code>Out[*]=</code>	$\mathcal{L}_v \partial_a f$
<code>In[*]:=</code>	<code>BD[la, LD[v, f[]]]</code> <code>% // LDtoCD</code>
<code>Out[*]=</code>	$\partial_a \mathcal{L}_v f$
<code>Out[*]=</code>	$\partial_a \mathcal{L}_v f$
<code>In[*]:=</code>	<code>BD[la, LD[v, f[]]]</code> <code>LDtoCD[%, BD]</code>
<code>Out[*]=</code>	$\partial_a \mathcal{L}_v f$
<code>Out[*]=</code>	$\partial_a v^b \partial_b f + \partial_a \partial_b f v^b$
<code>In[*]:=</code>	<code>CovD[la, LD[v, f[]]]</code> <code>% // LDtoCD</code>
<code>Out[*]=</code>	$D_a \mathcal{L}_v f$
<code>Out[*]=</code>	$D_a \mathcal{L}_v f$
<code>In[*]:=</code>	<code>CD[la, LD[v, something[]]]</code> <code>% // LDtoCD</code>
<code>Out[*]=</code>	$\nabla_a \mathcal{L}_v \text{something}[]$
<code>Out[*]=</code>	$\nabla_a \mathcal{L}_v \text{something}[]$

In[*]:=	CD[la, LD[v, f[]]]
Out[*]=	LDtoCD[%, CovD]
Out[*]=	$\nabla_a \mathcal{L}_v f$
Out[*]=	$\nabla_a \mathcal{L}_v f$
In[*]:=	CovD[la, LD[v, f[]]]
Out[*]=	LDtoCD[%, CovD]
Out[*]=	$D_a \mathcal{L}_v f$
Out[*]=	$D_a v^b D_b f + D_a D_b f v^b$

RiemannToGamma

In[*]:=	RiemannToGamma::usage
Out[*]=	RiemannToGamma[expr, curvRL, covD] expands curvature tensors (Riemann, Ricci, or Scalar) in 'expr' into derivatives of Christoffel symbols. 'curvRL' is a list of curvature tensors to expand. If 'curvRL' is omitted, all are expanded. If 'covD' is omitted, CD is used.

곡률 텐서 R_{abc}^d , Ricci 텐서 R_{ab} , Scalar 곡률 R 을 connection Γ_{ab}^c 로 바꾼다. 옵션으로 바꿀 텐서의 리스트와 공변 도함수의 이름이 있다.

$$R_{abc}^d \equiv -(D_a \Gamma_{bc}^d - D_b \Gamma_{ac}^d + \Gamma_{ap}^d \Gamma_{bc}^p - \Gamma_{bp}^d \Gamma_{ac}^p - f_{ab}^p \Gamma_{pc}^d) = R_{abc}^d \big|_{\text{Wald}} = R_{cba}^d \big|_{\text{MTW}}$$

Non-coordinate basis에서

In[*]:=	Off[CoordinateBasisFlag]
In[*]:=	RiemannToGamma[RiemannCD[la, lb, lc, ud]]
Out[*]=	$-\hat{\partial}_a \Gamma_{bc}^d + \hat{\partial}_b \Gamma_{ac}^d - \Gamma_{ae}^d \Gamma_{bc}^e + \Gamma_{ac}^e \Gamma_{be}^d + \Gamma_{ec}^d f_{ab}^e$

Coordinate basis에서

In[*]:=	On[CoordinateBasisFlag]
In[*]:=	RiemannToGamma[RiemannCD[la, lb, lc, ud]]
Out[*]=	$-\partial_a \Gamma_{bc}^d + \partial_b \Gamma_{ac}^d - \Gamma_{ae}^d \Gamma_{bc}^e + \Gamma_{ac}^e \Gamma_{be}^d$

In[*]:= **RiemannToGamma**[**RiemannCD**[**la**, **lb**, **lc**, **ld**]]

Out[*]=

$$-\Gamma_{aed} \Gamma_{bc}^e + \Gamma_{ac}^e \Gamma_{bed} - \partial_a \Gamma_{bc}^e g_{de} + \partial_b \Gamma_{ac}^e g_{de}$$

In[*]:= **RiemannToGamma**[**RiemannCD**[**la**, **lb**, **uc**, **ud**]]

Out[*]=

$$\Gamma_a^{ce} \Gamma_{be}^d - \Gamma_{ae}^d \Gamma_b^{ce} - \partial_a \Gamma_{be}^d g^{ce} + \partial_b \Gamma_{ae}^d g^{ce}$$

In[*]:= **RiemannToGamma**[**RiemannCD**[**la**, **lb**, **uc**, **ld**]]

Out[*]=

$$\partial_a \Gamma_{bd}^c - \partial_b \Gamma_{ad}^c + \Gamma_{ae}^c \Gamma_{bd}^e - \Gamma_{ad}^e \Gamma_{be}^c$$

In[*]:= **RiemannToGamma**[**RiemannCD**[**la**, **ub**, **lc**, **ud**]]

Out[*]=

$$-\Gamma_{ae}^d \Gamma_c^{be} + \Gamma_{ac}^e \Gamma_e^{bd} - \partial_a \Gamma_{ec}^d g^{be} + \partial_e \Gamma_{ac}^d g^{be}$$

In[*]:= **RiemannToGamma**[**RiemannCD**[**la**, **ub**, **lc**, **ld**]]

Out[*]=

$$-\Gamma_{aed} \Gamma_c^{be} + \Gamma_{ac}^e \Gamma_{ed}^b - \partial_a \Gamma_{ec}^f g_{df} g^{be} + \partial_e \Gamma_{ac}^f g_{df} g^{be}$$

In[*]:= **RiemannToGamma**[**RiemannCD**[**la**, **ub**, **uc**, **ud**]]

Out[*]=

$$\Gamma_a^{ce} \Gamma_e^{bd} - \Gamma_{ae}^d \Gamma^{bce} - \partial_a \Gamma_{ef}^d g^{be} g^{cf} + \partial_e \Gamma_{af}^d g^{be} g^{cf}$$

In[*]:= **RiemannToGamma**[**RiemannCD**[**la**, **ub**, **uc**, **ld**]]

Out[*]=

$$\Gamma_{ae}^c \Gamma_d^{be} - \Gamma_{ad}^e \Gamma_e^{bc} + \partial_a \Gamma_{ed}^c g^{be} - \partial_e \Gamma_{ad}^c g^{be}$$

In[*]:= **RiemannToGamma**[**RiemannCD**[**ua**, **lb**, **lc**, **ld**]]

Out[*]=

$$\Gamma_{bed} \Gamma_c^{ae} - \Gamma_{bc}^e \Gamma_{ed}^a + \partial_b \Gamma_{ec}^f g_{df} g^{ae} - \partial_e \Gamma_{bc}^f g_{df} g^{ae}$$

Connection 계수는 proper 텐서가 아니므로 텐서 연산자가 있는 경우는 바꾸지 않는다.

In[*]:= **CD**[**ld**, **RiemannCD**[**la**, **lb**, **lc**, **ud**]]

RiemannToGamma[%]

Out[*]=

$$\nabla_d R_{abc}^d$$

Out[*]=

$$\nabla_d R_{abc}^d$$

```
In[*]:= BD[ld, RiemannCD[la, lb, lc, ud]]
RiemannToGamma[%]
```

```
Out[*]=
```

$$\partial_d R_{abc}^d$$

```
Out[*]=
```

$$-\partial_d \partial_a \Gamma_{bc}^d + \partial_d \partial_b \Gamma_{ac}^d + \partial_d \Gamma_{be}^d \Gamma_{ac}^e - \partial_d \Gamma_{bc}^e \Gamma_{ae}^d - \partial_d \Gamma_{ae}^d \Gamma_{bc}^e + \partial_d \Gamma_{ac}^e \Gamma_{be}^d$$

Ricci 텐서를 connection 계수로 바꿀 때도 같은 함수를 이용한다.

```
In[*]:= RiemannToGamma[RicciCD[la, lb]]
```

```
Out[*]=
```

$$-\partial_a \Gamma_{cb}^c + \partial_c \Gamma_{ab}^c - \Gamma_{ac}^d \Gamma_{db}^c + \Gamma_{ab}^c \Gamma_{dc}^d$$

```
In[*]:= RiemannToGamma[RicciCD[la, ub]]
```

```
Out[*]=
```

$$\Gamma_a^{bc} \Gamma_{dc}^d - \Gamma_{ac}^d \Gamma_d^{bc} - \partial_a \Gamma_{cd}^c g^{bd} + \partial_c \Gamma_{ad}^c g^{bd}$$

```
In[*]:= RiemannToGamma[RicciCD[ua, lb]]
```

```
Out[*]=
```

$$\Gamma_{cd}^c \Gamma_b^{ad} - \Gamma_{cb}^d \Gamma_d^{ac} - \partial_c \Gamma_{db}^d g^{ac} + \partial_c \Gamma_{db}^c g^{ad}$$

```
In[*]:= RiemannToGamma[RicciCD[ua, ub]]
```

```
Out[*]=
```

$$-\Gamma_c^{bd} \Gamma_d^{ac} + \Gamma_{cd}^c \Gamma^{abd} - \partial_c \Gamma_{de}^d g^{ac} g^{be} + \partial_c \Gamma_{de}^c g^{ad} g^{be}$$

```
In[*]:= BD[lb, RicciCD[la, ub]]
```

```
RiemannToGamma[%]
```

```
Out[*]=
```

$$\partial_b R_a^b$$

```
Out[*]=
```

$$\partial_b g^{bc} \partial_d \Gamma_{ac}^d - \partial_a \Gamma_{bc}^b \partial_d g^{dc} - \partial_b \Gamma_c^{bd} \Gamma_{ad}^c + \partial_b \Gamma_{cd}^c \Gamma_a^{bd} + \partial_b \Gamma_a^{bc} \Gamma_{dc}^d - \partial_b \Gamma_{ac}^d \Gamma_d^{bc} - \partial_b \partial_a \Gamma_{cd}^c g^{bd} + \partial_b \partial_c \Gamma_{ad}^c g^{bd}$$

Ricci 텐서만 connection 계수로 바꾸려면 option에서 지정한다.

```
In[*]:= RiemannCD[ua, ub, lc, ld] × RicciCD[la, lb]
```

```
RiemannToGamma[%, {RicciCD}]
```

```
Out[*]=
```

$$R_{ab} R_{cd}^{ab}$$

```
Out[*]=
```

$$\Gamma_{ab}^e \Gamma_{fe}^f R_{cd}^{ab} - \partial_a \Gamma_{be}^b R_{cd}^{ae} - \Gamma_{ab}^e \Gamma_{ef}^b R_{cd}^{af} + \partial_a \Gamma_{be}^a R_{cd}^{be}$$

Scalar 곡률 텐서를 connection 계수로 바꿀 때도 같은 함수를 이용한다.

In[*]:= **RiemannToGamma[ScalarCD[]]**

Out[*]=

$$\Gamma_a^{ab} \Gamma_{cb}^c - \Gamma_{ab}^c \Gamma_c^{ab} - \partial_a \Gamma_{bc}^b g^{ac} + \partial_a \Gamma_{bc}^a g^{bc}$$

In[*]:= **BD[la, ScalarCD[]]**

RiemannToGamma[%]

Out[*]=

$$\partial_a R$$

Out[*]=

$$-\partial_a g^{bc} \partial_b \Gamma_{dc}^d + \partial_a g^{bc} \partial_d \Gamma_{bc}^d - \partial_a \Gamma_{bc}^{cd} \Gamma_{cd}^b + \partial_a \Gamma_{bc}^{bc} \Gamma_{dc}^d - \partial_a \Gamma_{bc}^d \Gamma_d^{bc} + \partial_a \Gamma_{bc}^b \Gamma_d^{dc} - \partial_a \partial_b \Gamma_{cd}^c g^{bd} + \partial_a \partial_b \Gamma_{cd}^b g^{cd}$$

Scalar 곡률만 connection 계수로 바꾸려면 option에서 지정한다.

In[*]:= **ScalarCD[] × RicciCD[la, lb]**

RiemannToGamma[%, {ScalarCD}]

Out[*]=

$$R_{ab} R$$

Out[*]=

$$\Gamma_c^{cd} \Gamma_{ed}^e R_{ab} - \Gamma_{cd}^e \Gamma_e^{cd} R_{ab} - \partial_c \Gamma_{de}^d g^{ce} R_{ab} + \partial_c \Gamma_{de}^c g^{de} R_{ab}$$

옵션으로 공변 도함수의 이름이 있다.

In[*]:= **SetDefaultKind[Capital]**

In[*]:= **DefDerivativeOperator[CovD, "D", Latin, TorsionFreeQ → False]**

In[*]:= **ScalarCD[] × RicciCovD[la, lb] × RiemannCovD[lc, ua, ld, ub]**

RiemannToGamma[%, {RicciCovD}, CovD]

Out[*]=

$$R[\mathcal{D}]_{ab} R[\mathcal{D}]_c^{a\ b} R$$

Out[*]=

$$\Gamma[\mathcal{D}]_{ab}^e \Gamma[\mathcal{D}]_{fe}^f R[\mathcal{D}]_c^{a\ b} R - \partial_a \Gamma[\mathcal{D}]_{be}^b R[\mathcal{D}]_c^{a\ d} R - \Gamma[\mathcal{D}]_{ab}^e \Gamma[\mathcal{D}]_{ef}^b R[\mathcal{D}]_c^{a\ f} R + \partial_a \Gamma[\mathcal{D}]_{be}^a R[\mathcal{D}]_c^{b\ d} R$$

In[*]:= **UndefDerivativeOperator[CovD];**

SetDefaultKind[Latin]

Riemann and Weyl

Weyl 텐서 C_{abc}^d 는 Riemann 텐서 R_{abc}^d 와 다음 관계가 있다:

$$C_{abcd} \equiv R_{abcd} - (\text{riemannSign}) \left(-\frac{1}{n-2} (g_{ac} R_{bd} - g_{ad} R_{bc} + g_{bd} R_{ac} - g_{bc} R_{ad}) + \frac{1}{(n-1)(n-2)} R (g_{ac} g_{bd} - g_{ad} g_{bc}) \right)$$

Components and Coordinate Transformations

Set and Clear Components

```
In[*]:= Tdefine[e, 1]; Tdefine[b, 1]; Tdefine[F, "-ba"]
```

SetComponents

```
In[*]:= SetComponents::usage
```

```
Out[*]=
```

SetComponents[tensor[indices], values] sets the components of a 'tensor'. 'values' can be a single expression for one component or a list/table for all components.

텐서의 성분값을 대입한다. 첫 번째 인자가 성분 텐서가 아닌 경우 두 번째 인자는 Table 표현이어야 한다.

See MathTensor by L. Parker and S. M. Christenson, chapter 4.

```
In[*]:= SetComponents[F[1, 1], 0];
SetComponents[F[2, 2], 0];
SetComponents[F[3, 3], 0];
SetComponents[F[4, 4], 0]
```

```
In[*]:= SetComponents[F[1, 4], -e[1]];
SetComponents[F[2, 4], -e[2]]; SetComponents[F[3, 4], -e[3]]
```

```
In[*]:= {F[1, 4], F[2, 4], F[3, 4]}
```

```
Out[*]=
```

$\{-e^1, -e^2, -e^3\}$

```
In[*]:= gaussLaw = BD[la, F[4, ua]]
```

```
Out[*]=
```

$\partial_a F^{4a}$

```
In[*]:= SumDum[gaussLaw, {1, 3}]
```

```
Out[*]=
```

$\partial_3 e^3 + \partial_2 e^2 + \partial_1 e^1$

```
In[*]:= SetComponents[F[1, 2], b[3]];
SetComponents[F[1, 3], -b[2]];
SetComponents[F[2, 3], b[1]]
```

```
In[*]:= {F[1, 2], F[1, 3], F[2, 3]}
Out[*]=
```

$$\{b^3, -b^2, b^1\}$$

Table 표현으로 텐서의 모든 성분값을 대입한다.

```
In[*]:= metric = {{1, 0, 0, 0}, {0, 1, 0, 0}, {0, 0, 1, 0}, {0, 0, 0, -1}};
SetComponents[Metricg[la, lb], metric]
```

```
In[*]:= Metricg[la, lc] × Metricg[lb, ld] × F[uc, ud]
Out[*]=
```

$$F^{cd} g_{ac} g_{bd}$$

```
In[*]:= Metricg[la, lc] × Metricg[lb, ld] × F[uc, ud]
dnF = Table[SumDum[%, {1, 4}], {la, -1, -4, -1}, {lb, -1, -4, -1}]
```

```
Out[*]=
F^{cd} g_{ac} g_{bd}
```

```
Out[*]=
{{0, b^3, -b^2, e^1}, {-b^3, 0, b^1, e^2}, {b^2, -b^1, 0, e^3}, {-e^1, -e^2, -e^3, 0}}
```

```
In[*]:= SetComponents[F[la, lb], dnF]
```

```
In[*]:= {F[-1, -4], F[-2, -3], F[-1, -2]}
Out[*]=
```

$$\{e^1, b^1, b^3\}$$

```
In[*]:= F[la, lb] × F[ua, ub]
SumDum[%, {1, 3}]
```

```
Out[*]=
F_{ab} F^{ab}
```

```
Out[*]=
2 b^{12} + 2 b^{22} + 2 b^{32}
```

SetComponents 함수를 이용하지 않고 함수를 정의하는 방법으로도 사용할 수 있다.

```
In[*]:= dnF2[la_, lb_] := SumDum[Metricg[la, lc] × Metricg[lb, ld] × F[uc, ud], {1, 4}]
```

```
In[*]:= {dnF2[-1, -4], dnF2[-2, -3], dnF2[-1, -2]}
Out[*]=
```

$$\{e^1, b^1, b^3\}$$

```
In[*]:= eq1[lb_, ua_, ub_] := BD[lb, F[ua, ub]] /; PairIndexQ[{lb, ub}];
eq2[la_, lb_, lc_] := BD[la, F[lb, lc]] + BD[lb, F[lc, la]] + BD[lc, F[la, lb]]
```

```
In[*]:= SumDum[eq1[lb, 1, ub], {1, 4}]
```

```
Out[*]=
```

$$-\partial_4 e^1 - \partial_3 b^2 + \partial_2 b^3$$

```
In[*]:= SumDum[eq1[lb, 2, ub], {1, 4}]
```

```
Out[*]=
```

$$-\partial_4 e^2 + \partial_3 b^1 - \partial_1 b^3$$

```
In[*]:= SumDum[eq1[lb, 4, ub], {1, 4}]
```

```
Out[*]=
```

$$\partial_3 e^3 + \partial_2 e^2 + \partial_1 e^1$$

```
In[*]:= eq2[-1, -2, -3]
```

```
Out[*]=
```

$$\partial_3 b^3 + \partial_2 b^2 + \partial_1 b^1$$

```
In[*]:= eq2[-1, -2, -4]
```

```
Out[*]=
```

$$\partial_4 b^3 - \partial_2 e^1 + \partial_1 e^2$$

ClearComponents

```
In[*]:= ClearComponents::usage
```

```
Out[*]=
```

ClearComponents[tensor[indices]] clears the components of a 'tensor'. If specific indices are given, only that component (and its symmetric equivalents) is cleared. If general indices are given, all components are cleared.

텐서의 성분값을 제거한다. 인자가 성분 텐서인 경우는 그 성분의 값만 제거하고, 아닌 경우는 모든 성분의 값을 제거한다.

```
In[*]:= ClearComponents[F[la, lb], {4, 4}]
```

```
In[*]:= {F[-2, -1], F[1, 3]}
```

```
Out[*]=
```

$$\{F_{21}, -b^2\}$$

```
In[*]:= {F[-1, -1], F[-1, -2]}
```

```
Out[*]=
```

$$\{F_{11}, F_{12}\}$$

```
In[*]:= ClearComponents[F[1, 3]]
```

```
In[*]:= {F[1, 3], F[2, 3]}
```

```
Out[*]= {F13, b1}
```

```
In[*]:= ClearComponents[F[ua, ub], {4, 4}]
```

```
In[*]:= F[2, 3]
```

```
Out[*]= F23
```

```
In[*]:= ClearComponents[Metricg[la, lb], {4, 4}] (* default *)
```

Check

```
In[*]:= SetDimension[4]
```

```
In[*]:= SetComponents[F[1, 5], some]
```

... Msg: The value of a component index 5 is larger than the dimension 4

```
In[*]:= SetComponents[F[1, -5], some]
```

... Msg: The value of a component index 5 is larger than the dimension 4

```
In[*]:= ClearDimension[]
```

```
In[*]:= SetComponents[F[la], {some1, some2}]
```

... Msg: invalid number of indices {la} with rank 2

```
In[*]:= SetComponents[F[la, lb], {some1, some2}]
```

... Msg: incompatible components for a rank 2

```
In[*]:= SetComponents[F[la, lb], {{1, 2, 3}, {4, 5, 6}}];  
Table[F[-i, -j], {i, 2}, {j, 3}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

```
( 1 2 3 )  
 ( 4 5 6 )
```

```
In[*]:= ClearComponents[F[la, lb]]
```

... Msg: need to set dimension for the index la

```
In[*]:= SetDimension[2]
ClearComponents[F[la, lb]]
Table[F[-i, -j], {i, 2}, {j, 3}] // MatrixForm
```

Out[*]//MatrixForm=

$$\begin{pmatrix} F_{11} & F_{12} & 3 \\ F_{21} & F_{22} & 6 \end{pmatrix}$$

```
In[*]:= SetDimension[3]
ClearComponents[F[la, lb]]
Table[F[-i, -j], {i, 2}, {j, 3}] // MatrixForm
```

Out[*]//MatrixForm=

$$\begin{pmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \end{pmatrix}$$

```
In[*]:= ClearDimension[]
SetComponents[F[la, lb], {{1, 2}, {3, 4}, {5, 6}}]
Table[F[-i, -j], {i, 3}, {j, 2}] // MatrixForm
```

Out[*]//MatrixForm=

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

```
In[*]:= ClearComponents[F[la, lb]]
```

... Msg: need to set dimension for the index la

```
In[*]:= ClearComponents[F[la, lb], {3, 2}]
Table[F[-i, -j], {i, 3}, {j, 2}] // MatrixForm
```

Out[*]//MatrixForm=

$$\begin{pmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \\ F_{31} & F_{32} \end{pmatrix}$$

```
In[*]:= SetDimension[2]
```

```
In[*]:= SetComponents[F[la, lb], {{1, 2}, {3, 4}}]
Table[F[-i, -j], {i, 2}, {j, 2}] // MatrixForm
```

Out[*]//MatrixForm=

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

```
In[*]:= ClearComponents[F[la, lb], {3, 2}]
```

... Msg: invalid input 3 for the dimension 2

```
In[*]:= ClearComponents[F[la, lb]]; (* default *)
ClearDimension[] (* default *)
```

Pushforward, Pullback, and PushTensor

Pushforward

In[*]:= **Pushforward::usage**

Out[*]=

Pushforward[fromT, fromCoSys, toCoSys, simpCmd] computes the pushforward of a vector or tensor 'fromT' from coordinate system 'fromCoSys' to 'toCoSys'. 'simpCmd' is an optional simplification function.

For $\{x\} \rightarrow \{y\}$

$$M_i^a = \frac{\partial y^a}{\partial x^i} \equiv J^a_i, \quad \xi'^a = M_i^a \xi^i = (M^T)^a_i \xi^i, \quad \xi' = M^T \xi$$

$$T'^{ab} = M_i^a M_j^b T^{ij} = (M^T)^a_i T^{ij} M_j^b, \quad T' = M^T T M$$

$$T'^{abc} = M_i^a M_j^b M_k^c T^{ijk}$$

Notation: [Jacobian matrix] = M^T

Maple's Differential Geometry: Pushforward Ex 1

In[*]:=

```
fromCoSys = {x, y}; toCoSys = {u, v};
forM = {u → Log[x^2 + y^2], v → x/y};
SequenceForm[MatrixForm[fromCoSys], "→", MatrixForm[toCoSys]]
```

Out[*]=

$$\begin{pmatrix} x \\ y \end{pmatrix} \rightarrow \begin{pmatrix} u \\ v \end{pmatrix}$$

In[*]:=

```
MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify
```

Out[*]=

$$\left\{ \left\{ \frac{2x}{x^2 + y^2}, \frac{1}{y} \right\}, \left\{ \frac{2y}{x^2 + y^2}, -\frac{x}{y^2} \right\} \right\}$$

In[*]:=

```
Transpose[MM].{a, b} // Simplify
```

Out[*]=

$$\left\{ \frac{2(ax + by)}{x^2 + y^2}, \frac{-bx + ay}{y^2} \right\}$$

In[*]:=

```
Pushforward[{a, b}, fromCoSys, toCoSys /. forM]
```

Out[*]=

$$\left\{ \frac{2(ax + by)}{x^2 + y^2}, \frac{-bx + ay}{y^2} \right\}$$

In[*]:= **PushTensor::usage**

Out[*]=

PushTensor[updnL, fromT, fromCoSys, toCoSys, forM, forN, simpCmd] transforms the tensor 'fromT' from 'fromCoSys' to 'toCoSys'. 'updnL' specifies the variance of indices. 'forM' and 'forN' are transformation rules.

In[*]:= **forN = {};**

PushTensor[{ua}, {a, b}, fromCoSys, toCoSys, forM, forN]

Out[*]=

$$\left\{ \frac{2 (a x + b y)}{x^2 + y^2}, \frac{-b x + a y}{y^2} \right\}$$

Maple's Differential Geometry: Pushforward Ex 4

In[*]:= **fromCoSys = {x, y}; toCoSys = {r, θ };**

leftRule = {x $\rightarrow$ r Cos[θ], y $\rightarrow$ r Sin[θ]};

forM = {r $\rightarrow$ $\sqrt{x^2 + y^2}$, $\theta \rightarrow$ ArcTan[$\frac{y}{\sqrt{x^2 + y^2}}$, $\frac{x}{\sqrt{x^2 + y^2}}$]};

SequenceForm[**MatrixForm**[fromCoSys], " $\rightarrow$ ", **MatrixForm**[toCoSys]]

Out[*]=

$$\begin{pmatrix} x \\ y \end{pmatrix} \rightarrow \begin{pmatrix} r \\ \theta \end{pmatrix}$$

In[*]:= **MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify**

Out[*]=

$$\left\{ \left\{ \frac{x}{\sqrt{x^2 + y^2}}, \frac{y}{x^2 + y^2} \right\}, \left\{ \frac{y}{\sqrt{x^2 + y^2}}, -\frac{x}{x^2 + y^2} \right\} \right\}$$

In[*]:= **Transpose[MM].{ $\frac{y}{x}$, -2} // Simplify**

Out[*]=

$$\left\{ -\frac{y}{\sqrt{x^2 + y^2}}, \frac{2 x^2 + y^2}{x^3 + x y^2} \right\}$$

In[*]:= **% /. leftRule // FullSimplify**

Out[*]=

$$\left\{ -\frac{r \sin[\theta]}{\sqrt{r^2}}, \frac{\cos[\theta] + \sec[\theta]}{r} \right\}$$


```
In[*]:= Pushforward[ $\left\{\frac{y}{x}, -2\right\}$ , fromCoSys, toCoSys /. forM]
% /. leftRule // FullSimplify
```

Out[*]=

$$\left\{-\frac{y}{\sqrt{x^2 + y^2}}, \frac{2x^2 + y^2}{x^3 + xy^2}\right\}$$

Out[*]=

$$\left\{-\frac{r \sin[\theta]}{\sqrt{r^2}}, \frac{\cos[\theta] + \sec[\theta]}{r}\right\}$$

```
In[*]:= forN = {};
PushTensor[{ua},  $\left\{\frac{y}{x}, -2\right\}$ , fromCoSys, toCoSys, forM, forN]
```

Out[*]=

$$\left\{-\frac{y}{\sqrt{x^2 + y^2}}, \frac{2x^2 + y^2}{x^3 + xy^2}\right\}$$

Maple's Differential Geometry: Pushforward Ex 5

Find the tangent vector to the curve $t \rightarrow \{x = t^2, y = t^3, z = t^3\}$.

```
In[*]:= fromCoSys = {t}; toCoSys = {x, y, z};
forM = {x → t^2, y → t^3, z → t^3};
SequenceForm[MatrixForm[fromCoSys], "→", MatrixForm[toCoSys]]
```

Out[*]=

$$(t) \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

```
In[*]:= MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify
```

Out[*]=

$$\{\{2t, 3t^2, 3t^2\}\}$$

```
In[*]:= Transpose[MM].{1} // Simplify
```

Out[*]=

$$\{2t, 3t^2, 3t^2\}$$

```
In[*]:= Pushforward[{1}, fromCoSys, toCoSys /. forM]
```

Out[*]=

$$\{2t, 3t^2, 3t^2\}$$

```
In[*]:= forN = {};
PushTensor[{ua}, {1}, fromCoSys, toCoSys, forM, forN]
```

```
Out[*]= {2 t, 3 t^2, 3 t^2}
```

Maple's Differential Geometry: Pushforward Ex 6

Find a basis for the tangent plane to the surface $z = x^3 - 3xy^2$.

```
In[*]:= fromCoSys = {x, y}; toCoSys = {x, y, z};
forM = {x → x, y → y, z → x^3 - 3 x y^2};
SequenceForm[MatrixForm[fromCoSys], "→", MatrixForm[toCoSys]]
```

```
Out[*]=  $\begin{pmatrix} x \\ y \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$ 
```

```
In[*]:= MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify
```

```
Out[*]= {{1, 0, 3 (x^2 - y^2)}, {0, 1, -6 x y}}
```

```
In[*]:= Transpose[MM].{1, 0} // Simplify
```

```
Out[*]= {1, 0, 3 (x^2 - y^2)}
```

```
In[*]:= Pushforward[{1, 0}, fromCoSys, toCoSys /. forM]
```

```
Out[*]= {1, 0, 3 (x^2 - y^2)}
```

```
In[*]:= forN = {};
PushTensor[{ua}, {1, 0}, fromCoSys, toCoSys, forM, forN]
```

```
Out[*]= {1, 0, 3 (x^2 - y^2)}
```

```
In[*]:= Transpose[MM].{0, 1} // Simplify
```

```
Out[*]= {0, 1, -6 x y}
```

```
In[*]:= Pushforward[{0, 1}, fromCoSys, toCoSys /. forM]
```

```
Out[*]= {0, 1, -6 x y}
```

In[*]:= **PushTensor**[{ua}, {0, 1}, fromCoSys, toCoSys, forM, {}]

Out[*]=

{0, 1, -6 x y}

Maple's Differential Geometry: Lesson 7, stereographic coordinates on S^2

In[*]:= **fromCoSys** = {u, v}; **toCoSys** = {x, y, z};
forM = $\left\{ x \rightarrow \frac{2u}{1+u^2+v^2}, y \rightarrow \frac{2v}{1+u^2+v^2}, z \rightarrow \frac{u^2+v^2-1}{1+u^2+v^2} \right\};$
SequenceForm[**MatrixForm**[fromCoSys], "→", **MatrixForm**[toCoSys]]

Out[*]=

$\begin{pmatrix} u \\ v \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$

In[*]:= **MM** = **Table**[**D**[toCoSys /. forM, fromCoSys[[i]], {i, Length[fromCoSys]}] // **Simplify**

Out[*]=

$\left\{ \left\{ \frac{2-2u^2+2v^2}{(1+u^2+v^2)^2}, -\frac{4uv}{(1+u^2+v^2)^2}, \frac{4u}{(1+u^2+v^2)^2} \right\}, \right.$
 $\left. \left\{ -\frac{4uv}{(1+u^2+v^2)^2}, \frac{2(1+u^2-v^2)}{(1+u^2+v^2)^2}, \frac{4v}{(1+u^2+v^2)^2} \right\} \right\}$

In[*]:= **Transpose**[**MM**].{a, b} // **Simplify**

Out[*]=

$\left\{ \frac{-4buv+2a(1-u^2+v^2)}{(1+u^2+v^2)^2}, \frac{2(-2auv+b(1+u^2-v^2))}{(1+u^2+v^2)^2}, \frac{4(au+bv)}{(1+u^2+v^2)^2} \right\}$

In[*]:= **Pushforward**[{a, b}, fromCoSys, toCoSys /. forM]

Out[*]=

$\left\{ \frac{-4buv+2a(1-u^2+v^2)}{(1+u^2+v^2)^2}, \frac{2(-2auv+b(1+u^2-v^2))}{(1+u^2+v^2)^2}, \frac{4(au+bv)}{(1+u^2+v^2)^2} \right\}$

In[*]:= **forN** = {};
PushTensor[{ua}, {a, b}, fromCoSys, toCoSys, forM, forN]

Out[*]=

$\left\{ \frac{-4buv+2a(1-u^2+v^2)}{(1+u^2+v^2)^2}, \frac{2(-2auv+b(1+u^2-v^2))}{(1+u^2+v^2)^2}, \frac{4(au+bv)}{(1+u^2+v^2)^2} \right\}$

Pullback

In[*]:= Pullback::usage

Out[*]=

Pullback[fromT, fromCoSys, toCoSys, simpCmd] computes the pullback of a covector or tensor 'fromT' from coordinate system 'fromCoSys' to 'toCoSys'. 'simpCmd' is an optional simplification function.

For $\{x\} \rightarrow \{y\}$

$$M_i^a = \frac{\partial y^a}{\partial x^i}, \quad \omega_i = M_i^a \omega'_a, \quad \omega = M \omega'$$

$$T_{ij} = M_i^a M_j^b T'_{ab} = M_i^a T'_{ab} (M^T)^b_j, \quad T = M T' M^T$$

$$T_{ijk} = M_i^a M_j^b M_k^c T'_{abc}$$

Notation: [Jacobian matrix] = M^T

Maple's Differential Geometry: Pullback Ex 1

In[*]:= fromCoSys = {x, y}; toCoSys = {u, v, w};
forM = {u → x + 2 y, v → 3 x + 4 y, w → 5 x + 6 y};
SequenceForm[MatrixForm[fromCoSys], "→", MatrixForm[toCoSys]]

Out[*]=

$$\begin{pmatrix} x \\ y \end{pmatrix} \rightarrow \begin{pmatrix} u \\ v \\ w \end{pmatrix}$$

In[*]:= MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify

Out[*]=

{{1, 3, 5}, {2, 4, 6}}

In[*]:= MM.{v, w, u} // Simplify

Out[*]=

{5 u + v + 3 w, 2 (3 u + v + 2 w)}

In[*]:= % /. forM // Simplify

Out[*]=

{23 x + 32 y, 32 x + 44 y}

In[*]:= Pullback[{v, w, u}, fromCoSys, toCoSys /. forM]
% /. forM // Simplify

Out[*]=

{5 u + v + 3 w, 2 (3 u + v + 2 w)}

Out[*]=

{23 x + 32 y, 32 x + 44 y}

PushTensor of 1-form for $\{y\} \rightarrow \{x\}$ corresponds to Pullback for $\{x\} \rightarrow \{y\}$:

```
In[*]:= forN = {};
PushTensor[{la}, {v, w, u}, toCoSys, fromCoSys, forN, forM]
% /. forM // Simplify
```

```
Out[*]:= {5 u + v + 3 w, 2 (3 u + v + 2 w)}
```

```
Out[*]:= {23 x + 32 y, 32 x + 44 y}
```

Maple's Differential Geometry: Pullback Ex 3

```
In[*]:= fromCoSys = {r, θ, ϕ}; toCoSys = {x, y, z};
forM = {x → r Cos[θ] Sin[ϕ], y → r Sin[θ] Sin[ϕ], z → r Cos[θ]};
SequenceForm[MatrixForm[fromCoSys], "→", MatrixForm[toCoSys]]
```

```
Out[*]:= 
$$\begin{pmatrix} r \\ \theta \\ \phi \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

```

```
In[*]:= MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify
```

```
Out[*]:= {{Cos[θ] Sin[ϕ], Sin[θ] Sin[ϕ], Cos[θ]},
{-r Sin[θ] Sin[ϕ], r Cos[θ] Sin[ϕ], -r Sin[θ]},
{r Cos[θ] Cos[ϕ], r Cos[ϕ] Sin[θ], 0}}
```

```
In[*]:= MM.{0, z, -y}, {-z, 0, x}, {y, -x, 0}}.Transpose[MM] // Simplify
```

```
Out[*]:= {{0, r Sin[ϕ] (-x + z Sin[ϕ]), r Cos[θ] Cos[ϕ] (y Cos[θ] - x Sin[θ])},
{r Sin[ϕ] (x - z Sin[ϕ]), 0,
-1/2 r^2 Cos[ϕ] (-x + x Cos[2 θ] + y Sin[2 θ] + 2 z Sin[ϕ])},
{-r Cos[θ] Cos[ϕ] (y Cos[θ] - x Sin[θ]),
1/2 r^2 Cos[ϕ] (-x + x Cos[2 θ] + y Sin[2 θ] + 2 z Sin[ϕ]), 0}}
```

```
In[*]:= % /. forM // Simplify
```

```
Out[*]:= {{0, 0, 0}, {0, 0, -r^3 Cos[θ] Cos[ϕ] Sin[ϕ]}, {0, r^3 Cos[θ] Cos[ϕ] Sin[ϕ], 0}}
```

```
In[*]:= Pullback[{{0, z, -y}, {-z, 0, x}, {y, -x, 0}}, fromCoSys, toCoSys /. forM]
% /. forM // Simplify
```

```
Out[*]=
```

$$\left\{ \begin{aligned} &\{0, r \sin[\phi] (-x + z \sin[\phi]), r \cos[\theta] \cos[\phi] (y \cos[\theta] - x \sin[\theta])\}, \\ &\left\{ r \sin[\phi] (x - z \sin[\phi]), 0, \right. \\ &\quad \left. -\frac{1}{2} r^2 \cos[\phi] (-x + x \cos[2\theta] + y \sin[2\theta] + 2z \sin[\phi]) \right\}, \\ &\left\{ -r \cos[\theta] \cos[\phi] (y \cos[\theta] - x \sin[\theta]), \right. \\ &\quad \left. \frac{1}{2} r^2 \cos[\phi] (-x + x \cos[2\theta] + y \sin[2\theta] + 2z \sin[\phi]), 0 \right\} \end{aligned} \right\}$$

```
Out[*]=
```

$$\{\{0, 0, 0\}, \{0, 0, -r^3 \cos[\theta] \cos[\phi] \sin[\phi]\}, \{0, r^3 \cos[\theta] \cos[\phi] \sin[\phi], 0\}\}$$

```
In[*]:= forN = {};
PushTensor[{la, lb}, {{0, z, -y}, {-z, 0, x}, {y, -x, 0}},
toCoSys, fromCoSys, forN, forM]
% /. forM // Simplify
```

```
Out[*]=
```

$$\left\{ \begin{aligned} &\{0, r \sin[\phi] (-x + z \sin[\phi]), r \cos[\theta] \cos[\phi] (y \cos[\theta] - x \sin[\theta])\}, \\ &\left\{ r \sin[\phi] (x - z \sin[\phi]), 0, \right. \\ &\quad \left. -\frac{1}{2} r^2 \cos[\phi] (-x + x \cos[2\theta] + y \sin[2\theta] + 2z \sin[\phi]) \right\}, \\ &\left\{ r \cos[\theta] \cos[\phi] (-y \cos[\theta] + x \sin[\theta]), \right. \\ &\quad \left. \frac{1}{2} r^2 \cos[\phi] (-x + x \cos[2\theta] + y \sin[2\theta] + 2z \sin[\phi]), 0 \right\} \end{aligned} \right\}$$

```
Out[*]=
```

$$\{\{0, 0, 0\}, \{0, 0, -r^3 \cos[\theta] \cos[\phi] \sin[\phi]\}, \{0, r^3 \cos[\theta] \cos[\phi] \sin[\phi], 0\}\}$$

Maple's Differential Geometry: Lesson 8

```
In[*]:= fromCoSys = {x, y, z}; toCoSys = {u, v};
forM = {u -> x^2 + y^2, v -> z^2 - y^2};
SequenceForm[MatrixForm[fromCoSys], "> ", MatrixForm[toCoSys]]
```

```
Out[*]=
```

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} u \\ v \end{pmatrix}$$

```
In[*]:= MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify
```

```
Out[*]=
```

$$\{\{2x, 0\}, \{2y, -2y\}, \{0, 2z\}\}$$

```
In[*]:= MM.{ $\frac{v}{2}, -\frac{u}{2}$ } // Simplify
% /. forM // Simplify
```

```
Out[*]= {v x, (u + v) y, -u z}
```

```
Out[*]= {x (-y2 + z2), y (x2 + z2), -((x2 + y2) z)}
```

```
In[*]:= Pullback[{ $\frac{v}{2}, -\frac{u}{2}$ }, fromCoSys, toCoSys /. forM]
% /. forM // Simplify
```

```
Out[*]= {v x, (u + v) y, -u z}
```

```
Out[*]= {x (-y2 + z2), y (x2 + z2), -((x2 + y2) z)}
```

```
In[*]:= forN = {};
PushTensor[{la}, { $\frac{v}{2}, -\frac{u}{2}$ }, toCoSys, fromCoSys, forN, forM]
% /. forM // Simplify
```

```
Out[*]= {v x, (u + v) y, -u z}
```

```
Out[*]= {x (-y2 + z2), y (x2 + z2), -((x2 + y2) z)}
```

Maple's Differential Geometry: Lesson 9, Ex 2. the induced metric of a torus

```
In[*]:= fromCoSys = {u, v}; toCoSys = {x, y, z};
forM = {x → (a + b Sin[v]) Cos[u], y → (a + b Sin[v]) Sin[u], z → b Cos[v]};
SequenceForm[MatrixForm[fromCoSys], "→", MatrixForm[toCoSys]]
```

```
Out[*]=  $\begin{pmatrix} u \\ v \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$ 
```

```
In[*]:= MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify
```

```
Out[*]= {{-Sin[u] (a + b Sin[v]), Cos[u] (a + b Sin[v]), 0},
{b Cos[u] Cos[v], b Cos[v] Sin[u], -b Sin[v]}}
```

```
In[*]:= MM.{1, 0, 0}, {0, 1, 0}, {0, 0, 1}.Transpose[MM] // Simplify
```

```
Out[*]= {{(a + b Sin[v])2, 0}, {0, b2}}
```

```
In[*]:= Pullback[{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}, fromCoSys, toCoSys /. forM]
Out[*]= {{(a + b Sin[v])^2, 0}, {0, b^2}}
```

```
In[*]:= forN = {};
PushTensor[{la, lb}, {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}},
  toCoSys, fromCoSys, forN, forM]
Out[*]= {{(a + b Sin[v])^2, 0}, {0, b^2}}
```

PushTensor

```
In[*]:= PushTensor::usage
Out[*]= PushTensor[updnL, fromT, fromCoSys, toCoSys,
  forM, forN, simpCmd] transforms the tensor 'fromT' from
  'fromCoSys' to 'toCoSys'. 'updnL' specifies the variance
  of indices. 'forM' and 'forN' are transformation rules.
```

For $\{x\} \rightarrow \{y\}$

$$\begin{aligned} M_i^a &= \frac{\partial y^a}{\partial x^i}, & N_a^i &= \frac{\partial x^i}{\partial y^a} \\ \xi'^a &= M_i^a \xi^i, & \xi^i &= M^T \xi' \\ \omega'^a &= N_a^i \omega_i, & \omega^i &= N \omega' \\ T'^a_b &= M_i^a N_b^j T^i_j, & T' &= M^T T N^T \\ T'^a_b &= N_a^i M_j^b T_i^j, & T' &= N T M \end{aligned}$$

Notation: [Jacobian matrix] = M^T

Maple's Differential Geometry: PushPullTensor Ex 1

```
In[*]:= fromCoSys = {x, y, z}; toCoSys = {u, v, w};
forM = {u -> x y, v -> y, w -> x z};
forN = {x -> u/v, y -> v, z -> w v/u};
SequenceForm[MatrixForm[fromCoSys], ">", MatrixForm[toCoSys]]
Out[*]= (x y z) -> (u v w)
```

```
In[*]:= MM = Table[D[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // Simplify
Out[*]= {{y, 0, z}, {x, 1, 0}, {0, 0, x}}
```


In[*]:= **NN = Table[D[fromCoSys /. forN, toCoSys[[i]], {i, Length[toCoSys]}] // Simplify**

Out[*]=

$$\left\{ \left\{ \frac{1}{v}, 0, -\frac{vw}{u^2} \right\}, \left\{ -\frac{u}{v^2}, 1, \frac{w}{u} \right\}, \left\{ 0, 0, \frac{v}{u} \right\} \right\}$$

In[*]:= **Transpose[MM].{1, 0, 0}, {0, 0, 0}, {0, 0, 0}.Transpose[NN] /. forN // Simplify**

Out[*]=

$$\left\{ \left\{ 1, -\frac{u}{v}, 0 \right\}, \{0, 0, 0\}, \left\{ \frac{w}{u}, -\frac{w}{v}, 0 \right\} \right\}$$

In[*]:= **PushTensor[{ua, lb}, {1, 0, 0}, {0, 0, 0}, {0, 0, 0},
fromCoSys, toCoSys, forM, forN] /. forN**

Out[*]=

$$\left\{ \left\{ 1, -\frac{u}{v}, 0 \right\}, \{0, 0, 0\}, \left\{ \frac{w}{u}, -\frac{w}{v}, 0 \right\} \right\}$$

In[*]:= **Transpose[MM].{1, 0, 0} /. forN**

Out[*]=

$$\left\{ v, 0, \frac{vw}{u} \right\}$$

In[*]:= **PushTensor[{ua}, {1, 0, 0}, fromCoSys, toCoSys, forM, forN] /. forN**

Out[*]=

$$\left\{ v, 0, \frac{vw}{u} \right\}$$

In[*]:= **Transpose[MM].{0, 1, 0} /. forN**

Out[*]=

$$\left\{ \frac{u}{v}, 1, 0 \right\}$$

In[*]:= **PushTensor[{ua}, {0, 1, 0}, fromCoSys, toCoSys, forM, forN] /. forN**

Out[*]=

$$\left\{ \frac{u}{v}, 1, 0 \right\}$$

In[*]:= **Transpose[MM].{0, y, 0} /. forN**

Out[*]=

$$\{u, v, 0\}$$

In[*]:= **PushTensor[{ua}, {0, y, 0}, fromCoSys, toCoSys, forM, forN] /. forN**

Out[*]=

$$\{u, v, 0\}$$

In[*]:= **MM**.{{0, 0, 1}, {0, 0, 0}, {0, 0, 0}}.Transpose[**MM**]

Out[*]=

$\{\{y z, 0, x y\}, \{x z, 0, x^2\}, \{0, 0, 0\}\}$

In[*]:= **Pullback**[[{0, 0, 1}, {0, 0, 0}, {0, 0, 0}], fromCoSys, toCoSys /. forM]

Out[*]=

$\{\{y z, 0, x y\}, \{x z, 0, x^2\}, \{0, 0, 0\}\}$

In[*]:= **PushTensor**[{la, lb},
{{0, 0, 1}, {0, 0, 0}, {0, 0, 0}}, toCoSys, fromCoSys, {}, forM]

Out[*]=

$\{\{y z, 0, x y\}, \{x z, 0, x^2\}, \{0, 0, 0\}\}$

Maple's Differential Geometry: PushPullTensor Ex 2

In[*]:= **fromCoSys** = {u, v, w}; **toCoSys** = {x1, x2, x3, x4};
forM = $\left\{x1 \rightarrow \frac{2 u}{u^2 + v^2 + w^2 + 1}, \right.$
 $\left. x2 \rightarrow \frac{2 v}{u^2 + v^2 + w^2 + 1}, x3 \rightarrow \frac{2 w}{u^2 + v^2 + w^2 + 1}, x4 \rightarrow \frac{u^2 + v^2 + w^2 - 1}{u^2 + v^2 + w^2 + 1} \right\};$
SequenceForm[**MatrixForm**[fromCoSys], "→", **MatrixForm**[toCoSys]]

Out[*]=

$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \rightarrow \begin{pmatrix} x1 \\ x2 \\ x3 \\ x4 \end{pmatrix}$

In[*]:= **MM** = **Table**[**D**[toCoSys /. forM, fromCoSys[[i]]], {i, Length[fromCoSys]}] // **Simplify**

Out[*]=

$\left\{ \left\{ \frac{2 (1 - u^2 + v^2 + w^2)}{(1 + u^2 + v^2 + w^2)^2}, -\frac{4 u v}{(1 + u^2 + v^2 + w^2)^2}, -\frac{4 u w}{(1 + u^2 + v^2 + w^2)^2}, \frac{4 u}{(1 + u^2 + v^2 + w^2)^2} \right\}, \right.$
 $\left\{ -\frac{4 u v}{(1 + u^2 + v^2 + w^2)^2}, \frac{2 (1 + u^2 - v^2 + w^2)}{(1 + u^2 + v^2 + w^2)^2}, -\frac{4 v w}{(1 + u^2 + v^2 + w^2)^2}, \frac{4 v}{(1 + u^2 + v^2 + w^2)^2} \right\},$
 $\left. \left\{ -\frac{4 u w}{(1 + u^2 + v^2 + w^2)^2}, -\frac{4 v w}{(1 + u^2 + v^2 + w^2)^2}, \frac{2 (1 + u^2 + v^2 - w^2)}{(1 + u^2 + v^2 + w^2)^2}, \frac{4 w}{(1 + u^2 + v^2 + w^2)^2} \right\} \right\}$

In[*]:= **MM**.{{1, 0, 0, 0}, {0, 1, 0, 0}, {0, 0, 1, 0}, {0, 0, 0, 1}}.Transpose[**MM**] // **Simplify**

Out[*]=

$\left\{ \left\{ \frac{4}{(1 + u^2 + v^2 + w^2)^2}, 0, 0 \right\}, \left\{ 0, \frac{4}{(1 + u^2 + v^2 + w^2)^2}, 0 \right\}, \left\{ 0, 0, \frac{4}{(1 + u^2 + v^2 + w^2)^2} \right\} \right\}$

```
In[*]:= Pullback[{{1, 0, 0, 0}, {0, 1, 0, 0}, {0, 0, 1, 0}, {0, 0, 0, 1}},
  fromCoSys, toCoSys /. forM]
```

```
Out[*]=
```

$$\left\{ \left\{ \frac{4}{(1+u^2+v^2+w^2)^2}, 0, 0 \right\}, \left\{ 0, \frac{4}{(1+u^2+v^2+w^2)^2}, 0 \right\}, \left\{ 0, 0, \frac{4}{(1+u^2+v^2+w^2)^2} \right\} \right\}$$

```
In[*]:= forN = {};
PushTensor[{la, lb}, {{1, 0, 0, 0}, {0, 1, 0, 0}, {0, 0, 1, 0}, {0, 0, 0, 1}},
  toCoSys, fromCoSys, forN, forM]
```

```
Out[*]=
```

$$\left\{ \left\{ \frac{4}{(1+u^2+v^2+w^2)^2}, 0, 0 \right\}, \left\{ 0, \frac{4}{(1+u^2+v^2+w^2)^2}, 0 \right\}, \left\{ 0, 0, \frac{4}{(1+u^2+v^2+w^2)^2} \right\} \right\}$$

Ttransform for diffeomorphic transformations

```
In[8]:= Ttransform::usage
```

```
Out[8]= Ttransform[leftT, rightT, leftCoSys, rightCoSys, simpCmd]
  transforms the components of tensors between two coordinate
  systems. 'simpCmd' is an optional simplification function.
```

좌표 변환에 따른 텐서 성분의 변환 값을 얻는다. 첫 번째와 두 번째 인자는 텐서들이고, 세 번째와 네 번째 인자는 두 좌표계에 대한 리스트 표현이다. 옵션으로 simplification을 위한 함수가 올 수 있다. 첫 번째와 두 번째의 텐서 인자는 성분값을 알고 있는 경우는 텐서 표현, 변환된 값을 알고 싶은 경우는 텐서의 이름을 사용한다.

For $\{x\} \rightarrow \{y\}$

$$M_a^b = \frac{\partial y^b}{\partial x^a}, \quad N_a^b = \frac{\partial x^b}{\partial y^a}$$

$$(T')^{ab} = M_c^a M_d^b T^{cd}, \quad \omega'_{ab} = N_a^c N_b^d \omega_{cd}$$

$$T' = M^T T M, \quad \omega' = N \omega N^T$$

Notation: [Jacobian matrix] = M^T

For diffeomorphism, $N = M^{-1}$.

```
In[9]:= Tdefine[leftT, "ba"]; Tdefine[rightT, "ba"]
```

```
In[10]:= SetComponents[rightT[la, lb], {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}];
SetComponents[rightT[ua, ub], {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}]
```

Familiar Calculation

```
In[12]:= leftCoSys = {r,  $\theta$ ,  $\phi$ }; rightCoSys = {x, y, z};
forM = {x  $\rightarrow$  r Sin[ $\theta$ ] Cos[ $\phi$ ], y  $\rightarrow$  r Sin[ $\theta$ ] Sin[ $\phi$ ], z  $\rightarrow$  r Cos[ $\theta$ ]};
SequenceForm[MatrixForm[leftCoSys], " $\rightarrow$ ", MatrixForm[rightCoSys]]
```

```
Out[14]=
```

$$\begin{pmatrix} r \\ \theta \\ \phi \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

```
In[15]:= MM = Table[D[rightCoSys /. forM, leftCoSys[[i]]], {i, Length[leftCoSys]}]
```

```
Out[15]=
```

$$\begin{Bmatrix} \{\text{Cos}[\phi] \text{Sin}[\theta], \text{Sin}[\theta] \text{Sin}[\phi], \text{Cos}[\theta]\}, \\ \{r \text{Cos}[\theta] \text{Cos}[\phi], r \text{Cos}[\theta] \text{Sin}[\phi], -r \text{Sin}[\theta]\}, \\ \{-r \text{Sin}[\theta] \text{Sin}[\phi], r \text{Cos}[\phi] \text{Sin}[\theta], 0\} \end{Bmatrix}$$

$\text{leftT}_{ab} \leftarrow \text{rightT}_{ab}$

```
In[16]:= MM.{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}.Transpose[MM] // Simplify
```

```
Out[16]=
```

$$\{\{1, 0, 0\}, \{0, r^2, 0\}, \{0, 0, r^2 \text{Sin}[\theta]^2\}\}$$

```
In[17]:= Ttransform[leftT, rightT[la, lb], leftCoSys, rightCoSys /. forM, Simplify];
Table[leftT[-i, -j], {i, 3}, {j, 3}]
```

```
Out[18]=
```

$$\{\{1, 0, 0\}, \{0, r^2, 0\}, \{0, 0, r^2 \text{Sin}[\theta]^2\}\}$$

$\text{leftT}_{ab} \rightarrow \text{rightT}_{ab}$

```
In[19]:= Inverse[MM].{{1, 0, 0}, {0, r^2, 0}, {0, 0, r^2 Sin[ $\theta$ ]^2}}.
Transpose[Inverse[MM]] // Simplify
```

```
Out[19]=
```

$$\{\{1, 0, 0\}, \{0, 1, 0\}, \{0, 0, 1\}\}$$

```
In[20]:= Ttransform[leftT[la, lb], rightT, leftCoSys, rightCoSys /. forM, Simplify];
Table[rightT[-i, -j], {i, 3}, {j, 3}]
```

```
Out[21]=
```

$$\{\{1, 0, 0\}, \{0, 1, 0\}, \{0, 0, 1\}\}$$

$\text{leftT}^{ab} \leftarrow \text{rightT}^{ab}$

```
In[22]:= Transpose[Inverse[MM]].{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}.Inverse[MM] // Simplify
```

```
Out[22]=
```

$$\{\{1, 0, 0\}, \{0, \frac{1}{r^2}, 0\}, \{0, 0, \frac{\text{Csc}[\theta]^2}{r^2}\}\}$$

```
In[23]:= Ttransform[leftT, rightT[ua, ub], leftCoSys, rightCoSys /. forM, Simplify];
Table[leftT[i, j], {i, 3}, {j, 3}]
```

Out[24]=

$$\left\{ \{1, 0, 0\}, \left\{0, \frac{1}{r^2}, 0\right\}, \left\{0, 0, \frac{\text{Csc}[\theta]^2}{r^2}\right\} \right\}$$

$\text{leftT}^{ab} \rightarrow \text{rightT}^{ab}$

```
In[25]:= Transpose[MM].{ {1, 0, 0}, {0, 1/r^2, 0}, {0, 0, Csc[θ]^2/r^2} }.MM // Simplify
```

Out[25]=

$$\left\{ \{1, 0, 0\}, \{0, 1, 0\}, \{0, 0, 1\} \right\}$$

```
In[26]:= Ttransform[leftT[ua, ub], rightT, leftCoSys, rightCoSys /. forM, Simplify];
Table[rightT[i, j], {i, 3}, {j, 3}]
```

Out[27]=

$$\left\{ \{1, 0, 0\}, \{0, 1, 0\}, \{0, 0, 1\} \right\}$$

Simple Shear

See MathTensor, section 8.2

```
In[28]:= leftCoSys = {x, y, z}; rightCoSys = {x + K y, y, z};
SequenceForm[MatrixForm[leftCoSys], ">", MatrixForm[rightCoSys]]
```

Out[29]=

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} x + K y \\ y \\ z \end{pmatrix}$$

```
In[30]:= MM = Table[D[rightCoSys, leftCoSys[[i]]], {i, 3}]
```

Out[30]=

$$\left\{ \{1, 0, 0\}, \{K, 1, 0\}, \{0, 0, 1\} \right\}$$

$\text{leftT}_{ab} \leftarrow \text{rightT}_{ab}$

```
In[31]:= SetComponents[rightT[la, lb], {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}];
```

```
In[32]:= MM.{ {1, 0, 0}, {0, 1, 0}, {0, 0, 1} }.Transpose[MM]
```

Out[32]=

$$\left\{ \{1, K, 0\}, \{K, 1 + K^2, 0\}, \{0, 0, 1\} \right\}$$

```
In[33]:= Ttransform[leftT, rightT[la, lb], leftCoSys, rightCoSys];
Table[leftT[-i, -j], {i, 3}, {j, 3}]
```

Out[34]=

$$\left\{ \{1, K, 0\}, \{K, 1 + K^2, 0\}, \{0, 0, 1\} \right\}$$

$\text{leftT}_{ab} \rightarrow \text{rightT}_{ab}$

In[35]:= `Inverse[MM].{{1, K, 0}, {K, 1 + K2, 0}, {0, 0, 1}}.Transpose[Inverse[MM]]`

Out[35]=
`{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}`

In[36]:= `Ttransform[leftT[la, lb], rightT, leftCoSys, rightCoSys];
Table[rightT[-i, -j], {i, 3}, {j, 3}]`

Out[37]=
`{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}`

 $\text{leftT}^{ab} \leftarrow \text{rightT}^{ab}$

In[38]:= `SetComponents[rightT[ua, ub], {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}];`

In[39]:= `Transpose[Inverse[MM]].{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}.Inverse[MM]`

Out[39]=
`{{1 + K2, -K, 0}, {-K, 1, 0}, {0, 0, 1}}`

In[40]:= `Ttransform[leftT, rightT[ua, ub], leftCoSys, rightCoSys];
Table[leftT[i, j], {i, 3}, {j, 3}]`

Out[41]=
`{{1 + K2, -K, 0}, {-K, 1, 0}, {0, 0, 1}}`

 $\text{leftT}^{ab} \rightarrow \text{rightT}^{ab}$

In[42]:= `Transpose[MM].{{1 + K2, -K, 0}, {-K, 1, 0}, {0, 0, 1}}.MM // Simplify`

Out[42]=
`{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}`

In[45]:= `Ttransform[leftT[ua, ub], rightT, leftCoSys, rightCoSys];
Table[rightT[i, j], {i, 3}, {j, 3}]`

Out[46]=
`{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}`

Misc. Commands

CommutatorVectors

In[*]:= **CommutatorVectors::usage**
Out[*]=

CommutatorVectors[vecList, kind] computes the commutator of vector fields given in 'vecList' for the manifold 'kind'. The default kind is DefaultKind.

벡터들의 commutator를 얻는다. 인자로는 벡터의 리스트 또는 두 개의 벡터이다. 이 함수를 사용하려면 SetCoordinates 함수나 InitCTensor 함수를 이용하여 좌표계를 초기화하여야 한다. 옵션으로 simplification을 위한 함수가 있다.

In[*]:= **SetCoordinates[{t, x}]**

In[*]:= **v1 = {t, x}; v2 = {x, t}; v3 = {t, x²};**
CommutatorVectors[{v1, v2, v3}]

$[V_1, V_3] = \{0, x^2\}$

$[V_2, V_3] = \{x - x^2, -t + 2tx\}$

In[*]:= **CommutatorVectors[v1, v2]**
Out[*]=

$\{0, 0\}$

In[*]:= **CommutatorVectors[v1, v3]**
Out[*]=

$\{0, x^2\}$

In[*]:= **ClearCoordinates[] (* default *)**

LineElement

In[*]:= **LineElement::usage**
Out[*]=

LineElement[coSys, metric, simpCmd] generates the line element 'ds^2' for a given coordinate system 'coSys' and metric matrix. 'simpCmd' is an optional simplification function.

좌표계와 metric의 입력을 받아 line element를 표시해 준다. 옵션으로 simplification 함수가 있고, default는 Together이다.

```
In[*]:= coSys = {t, r,  $\theta$ ,  $\phi$ };
metric =
  {{-e2a[t,r], 0, 0, 0}, {0, e2b[t,r], 0, 0}, {0, 0, r2, 0}, {0, 0, 0, r2 Sin[ $\theta$ ]2}};
LineElement[coSys, metric]
```

```
Out[*]=
```

$$ds^2 = e^{2b[t,r]} dr^2 - e^{2a[t,r]} dt^2 + r^2 d\theta^2 + r^2 d\phi^2 \sin[\theta]^2$$

SetConstantMetric, ClearConstantMetric (Experimental)

인자가 없는 경우

```
In[*]:= SetConstantMetric::usage
```

```
Out[*]=
```

SetConstantMetric[diag, coSys, kind] sets up a flat spacetime with a constant metric. 'diag' can be a list of diagonal elements (e.g., {-1, 1, 1, 1}) or a full metric matrix. 'coSys' is the coordinate system. The default kind is DefaultKind.

```
In[*]:= ClearConstantMetric::usage
```

```
Out[*]=
```

ClearConstantMetric[kind] clears the definitions for a flat spacetime with a constant metric for the specified 'kind'. The default kind is DefaultKind.

DefaultKind의 Metricg를 데카르트 좌표계에서의 상수 계량 텐서로 설정한다:

```
In[*]:= SetConstantMetric[]
```

```
In[*]:= MetricCompatibleQ[BD, Metricg]
```

```
Out[*]=
```

True

```
In[*]:= SP`constantMetricQ[DefaultKind]
```

```
Out[*]=
```

True

```
In[*]:= BD[la, Metricg[lb, lc]]
```

```
Out[*]=
```

0


```
In[*]:= {GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}
```

```
Out[*]= {0, 0, 0, 0}
```

DefaultKind의 상수 계량 텐서 설정을 해제한다:

```
In[*]:= ClearConstantMetric[]
```

```
In[*]:= MetricCompatibleQ[BD, Metricg]
```

```
Out[*]= False
```

```
In[*]:= SP`constantMetricQ[DefaultKind]
```

```
Out[*]= False
```

```
In[*]:= BD[la, Metricg[lb, lc]]
```

```
Out[*]=  $\partial_a g_{bc}$ 
```

```
In[*]:= {GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}
```

```
Out[*]= { $\Gamma_{abc}$ ,  $R_{abcd}$ ,  $R_{ab}$ ,  $R$ }
```

인자가 상수 벡터인 경우

DefaultKind에서 상수(+1 또는 -1) 벡터 (계량 텐서의 대각선 표현)으로부터 시공간 차원과 Sig, 좌표계, 계량 텐서 성분, 볼륨 폼 성분을 결정하고 Metricg를 상수 계량 텐서로 설정한다:

```
In[*]:= SetConstantMetric[{-1, 1}]
```

In[*]:= Show[DefaultKind]

Out[*]//TableForm=

AutoFlag	True
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True
-----	-----
Kind	Latin
Dimension	2
Sig	1
Coordinates	x1
	x2
CoordinateBasisQ	True
EvaluateBDFlag	False

In[*]:= Table[Metricg[-i, -j], {i, 2}, {j, 2}] // MatrixForm

Out[*]//MatrixForm=

$$\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

In[*]:= {Epsilon[-1, -2], Epsilon[-2, -1], Epsilon[1, 2], Epsilon[2, 1]}

Out[*]=

{1, -1, -1, 1}

In[*]:= {Epsilon[-1, -2, -3], Epsilon[-1, 2], Epsilon[1, 2, 3, 4]}

Out[*]=

$$\{\epsilon_{123}, \epsilon_1^2, \epsilon^{1234}\}$$

In[*]:= MetricCompatibleQ[BD, Metricg]

Out[*]=

True

In[*]:= SP`constantMetricQ[DefaultKind]

Out[*]=

True

In[*]:= BD[la, Metricg[lb, lc]]

Out[*]=

0

```
In[*]:= {GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}
```

```
Out[*]= {0, 0, 0, 0}
```

DefaultKind의 상수 계량 텐서 설정을 해제한다:

```
In[*]:= ClearConstantMetric[]
```

```
In[*]:= Show[DefaultKind]
```

```
Out[*]//TableForm=
```

AutoFlag	True
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True
-----	-----
Kind	Latin
Dimension	Any
Sig	Any
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= Table[Metricg[-i, -j], {i, 2}, {j, 2}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} g_{11} & g_{12} \\ g_{21} & g_{22} \end{pmatrix}$$

```
In[*]:= {Epsilon[-1, -2], Epsilon[1, 2]}
```

```
Out[*]=
```

$$\{\epsilon_{12}, \epsilon^{12}\}$$

```
In[*]:= MetricCompatibleQ[BD, Metricg]
```

```
Out[*]=
```

False

```
In[*]:= SP`constantMetricQ[DefaultKind]
```

```
Out[*]=
```

False

```
In[*]:= BD[la, Metricg[lb, lc]]
```

```
Out[*]=
```

$$\partial_a g_{bc}$$

```
In[*]:= {GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}
```

```
Out[*]= { $\Gamma_{abc}$ ,  $R_{abcd}$ ,  $R_{ab}$ ,  $R$ }
```

인자가 상수 벡터와 좌표계인 경우

DefaultKind에서 상수 벡터와 좌표계를 입력으로 받아 시공간 차원과 Sig, 계량 텐서 성분, 볼륨 폼 성분을 결정하고 Metricg를 상수 계량 텐서로 설정한다:

```
In[*]:= SetConstantMetric[{-1, 1, 1}, {t, x, y}]
```

```
In[*]:= Show[DefaultKind]
```

```
Out[*]//TableForm=
```

AutoFlag	True
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True
-----	-----
Kind	Latin
Dimension	3
Sig	1
Coordinates	t
	x
	y
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= Table[Metricg[-i, -j], {i, 3}, {j, 3}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
In[*]:= {Epsilon[-1, -2, -3], Epsilon[-2, -1, 3], Epsilon[1, 2, 3], Epsilon[2, 1, 3]}
```

```
Out[*]=
```

$$\{1, \epsilon_{21}^3, -1, 1\}$$

```
In[*]:= {Epsilon[-1, 2], Epsilon[1, 2, 3, 4]}
```

```
Out[*]=
```

$$\{\epsilon_1^2, \epsilon^{1234}\}$$

In[*]:= **SP`constantMetricQ[DefaultKind]**

Out[*]=

True

In[*]:= **BD[la, Metricg[lb, lc]]**

Out[*]=

0

In[*]:= **{GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}**

Out[*]=

{0, 0, 0, 0}

DefaultKind의 상수 계량 텐서 설정을 해제한다:

In[*]:= **ClearConstantMetric[]**

In[*]:= **Show[DefaultKind]**

Out[*]//TableForm=

AutoFlag	True
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True
-----	-----
Kind	Latin
Dimension	Any
Sig	Any
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

In[*]:= **Table[Metricg[-i, -j], {i, 3}, {j, 3}] // MatrixForm**

Out[*]//MatrixForm=

$$\begin{pmatrix} g_{11} & g_{12} & g_{13} \\ g_{21} & g_{22} & g_{23} \\ g_{31} & g_{32} & g_{33} \end{pmatrix}$$

In[*]:= **{Epsilon[-1, -2, -3], Epsilon[1, 2, 3]}**

Out[*]=

$\{\epsilon_{123}, \epsilon^{123}\}$

```
In[*]:= SP`constantMetricQ[DefaultKind]
```

```
Out[*]=
```

```
False
```

```
In[*]:= BD[la, Metricg[lb, lc]]
```

```
Out[*]=
```

```
 $\partial_a g_{bc}$ 
```

```
In[*]:= {GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}
```

```
Out[*]=
```

```
 $\{\Gamma_{abc}, R_{abcd}, R_{ab}, R\}$ 
```

상수 벡터와 좌표계의 차원이 일치해야 한다:

```
In[*]:= SetConstantMetric[{-1, 1, 1}, {t, x}]
```

```
Msg: incompatible arguments: mGRG`STensor`Private`vec and {t, x}
```

```
Out[*]=
```

```
$Failed
```

인자가 수치 행렬인 경우

기준 벡터가 Null 벡터인 경우:

```
In[*]:= SetConstantMetric[{{0, -1}, {-1, 0}}]
```

```
In[*]:= Show[DefaultKind]
```

```
Out[*]//TableForm=
```

AutoFlag	True
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True
-----	-----
Kind	Latin
Dimension	2
Sig	1
Coordinates	x1 x2
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= Table[Metricg[-i, -j], {i, 2}, {j, 2}] // MatrixForm
```

```
Out[*] // MatrixForm =
```

$$\begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix}$$

```
In[*]:= {Epsilon[-1, -2], Epsilon[-2, -1], Epsilon[1, 2], Epsilon[2, 1]}
```

```
Out[*] =
```

```
{1, -1, -1, 1}
```

```
In[*]:= {Epsilon[-1, -2, -3], Epsilon[-1, 2], Epsilon[1, 2, 3, 4]}
```

```
Out[*] =
```

```
{ $\epsilon_{123}$ ,  $\epsilon_1^2$ ,  $\epsilon^{1234}$ }
```

```
In[*]:= SP`constantMetricQ[DefaultKind]
```

```
Out[*] =
```

```
True
```

```
In[*]:= BD[la, Metricg[lb, lc]]
```

```
Out[*] =
```

```
0
```

```
In[*]:= {GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}
```

```
Out[*] =
```

```
{0, 0, 0, 0}
```

DefaultKind의 상수 계량 텐서 설정을 해제한다:

```
In[*]:= ClearConstantMetric[]
```

```
In[*]:= Show[DefaultKind]
```

```
Out[*] // TableForm =
```

AutoFlag	True
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True
-----	-----
Kind	Latin
Dimension	Any
Sig	Any
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= Table[Metricg[-i, -j], {i, 2}, {j, 2}] // MatrixForm
```

```
Out[*] // MatrixForm =
```

$$\begin{pmatrix} g_{11} & g_{12} \\ g_{21} & g_{22} \end{pmatrix}$$

```
In[*]:= {Epsilon[-1, -2], Epsilon[1, 2]}
```

```
Out[*] =
```

$$\{\epsilon_{12}, \epsilon^{12}\}$$

```
In[*]:= SP`constantMetricQ[DefaultKind]
```

```
Out[*] =
```

False

```
In[*]:= BD[la, Metricg[lb, lc]]
```

```
Out[*] =
```

$$\partial_a g_{bc}$$

```
In[*]:= {GammaCD[la, lb, lc], RiemannCD[la, lb, lc, ld], RicciCD[la, lb], ScalarCD[]}
```

```
Out[*] =
```

$$\{\Gamma_{abc}, R_{abcd}, R_{ab}, R\}$$

마지막 인자가 Kind인 경우

계량 텐서 공간이어야 한다:

```
In[*]:= SetConstantMetric[{-1, 1, 1}, Greek]
```

Msg: Greek is not a metric space.

```
Out[*] =
```

\$Failed

```
In[*]:= DefMetric[eta, "η", Greek]
```

좌표 기준 벡터이어야 한다:

```
In[*]:= SetConstantMetric[{-1, 1, 1}, Greek]
```

Msg: Greek is not in coordinate basis.

```
Out[*] =
```

\$Failed

```
In[*]:= On[CoordinateBasisFlag[Greek]]
```

인자가 상수 벡터, 좌표계, kind인 경우

```
In[*]:= SetConstantMetric[{-1, 1, 1}, {t, x, y}, Greek]
```



```
In[*]:= Show[Greek]
```

```
Out[*]//TableForm=
```

Kind	Greek
Dimension	3
Sig	1
Coordinates	t x y
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= Table[eta[-i, -j], {i, 3}, {j, 3}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
In[*]:= {EpsilonGreek[-1, -2, -3], EpsilonGreek[1, 2, 3]}
```

```
Out[*]=
```

```
{1, -1}
```

```
In[*]:= SP`constantMetricQ[Greek]
```

```
Out[*]=
```

```
True
```

```
In[*]:= BD[lμ, eta[uv, uρ]]
```

```
Out[*]=
```

```
0
```

Greek kind의 상수 계량 텐서 설정을 해제한다:

```
In[*]:= ClearConstantMetric[Greek]
```

```
In[*]:= Show[Greek] (* NB: CoordinateBasisQ == True *)
```

```
Out[*]//TableForm=
```

Kind	Greek
Dimension	Any
Sig	Any
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= Table[eta[-i, -j], {i, 3}, {j, 3}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} \eta_{11} & \eta_{12} & \eta_{13} \\ \eta_{21} & \eta_{22} & \eta_{23} \\ \eta_{31} & \eta_{32} & \eta_{33} \end{pmatrix}$$

```
In[*]:= {EpsilonGreek[-1, -2, -3], EpsilonGreek[1, 2, 3]}
```

```
Out[*]= {∈[Greek]123, ∈[Greek]123}
```

```
In[*]:= SP`constantMetricQ[Greek]
```

```
Out[*]= False
```

```
In[*]:= BD[lμ, eta[uv, up]]
```

```
Out[*]= ∂μηvp
```

공변 도함수를 정의한다:

```
In[*]:= DefDerivativeOperator[CovD, "D", Greek];
SetMetricCompatible[CovD, eta]
```

```
In[*]:= SetConstantMetric[{-1, 1, 1}, {t, x, y}, Greek]
```

```
In[*]:= {GammaCovD[lμ, lv, lp], RiemannCovD[lμ, lv, lp, lσ],
RicciCovD[lμ, lv], ScalarCovD[]}
```

```
Out[*]= {0, 0, 0, 0}
```

```
In[*]:= ClearConstantMetric[Greek]
```

```
In[*]:= {GammaCovD[lμ, lv, lp], RiemannCovD[lμ, lv, lp, lσ],
RicciCovD[lμ, lv], ScalarCovD[]}
```

```
Out[*]= {Γ[D]μνρ, R[D]μνρσ, R[D]μν, R[D]}
```

Special Relativity (Minkowski SpaceTime)

```
In[*]:= SetIndices[ToString /@ {a, b, c, d, e, f, g}, Latin];
DefKind[LatinSpace, ToString /@ {i, j, k, l, m, n}]
```

```
In[*]:= DefMetric[η, Greek]
```

⋯ Msg: eta will be replaced with a new metric η

```
In[*]:= On[CoordinateBasisFlag[Greek]]
```

```
In[*]:= SetConstantMetric[{1, 1, 1, -1}, {x, y, z, t}, Greek]
```

```
In[*]:= Table[η[-i, -j], {i, 4}, {j, 4}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

```
In[*]:= Show[Greek]
```

```
Out[*]//TableForm=
```

Kind	Greek
Dimension	4
Sig	1
	x
Coordinates	y
	z
	t
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= DefKind[Zero, {"0"}];
zeroRule = {l0 → -4, u0 → 4};
```

```
In[*]:= Tdefine[x, 1]
```

```
In[*]:= {x[l0], x[-1], x[-2], x[-3], x[u0], x[1], x[2], x[3]}
```

```
Out[*]=
```

$$\{x_0, x_1, x_2, x_3, x^0, x^1, x^2, x^3\}$$

```
In[*]:= % /. zeroRule
```

```
Out[*]=
```

$$\{x_4, x_1, x_2, x_3, x^4, x^1, x^2, x^3\}$$

```
In[*]:= UndefKind[Zero];
UndefKind[LatinSpace];
SetIndices[Alphabet[], Latin] (* default *)
```